

A Project Report
on
**FAKEDTECT: AI-DRIVEN MULTIMODAL FAKE NEWS
AND DEEPFAKE DETECTION SYSTEM**

Submitted to
OSMANIA UNIVERSITY
For the partial fulfilment of Academic requirement for the award of
MASTER OF SCIENCE
in
COMPUTER SCIENCE

By
RAVALKOL PRAVALIKA
H.T No: 1007-24-504-011



**UNIVERSITY COLLEGE OF SCIENCE, OSMANIA
UNIVERSITY, HYDERABAD – 500007**
M.Sc. 2024 - 2026



Department of Mathematics
University College of Science
Osmania University
HYDERABAD - 500007
Phone: 040 27095083

CERTIFICATE

This is to certify that **RAVALKOL PRAVALIKA** bearing Hall Ticket No. **1007-24-504-011**, a student of this college pursuing **Master of Science in Computer Science**, has successfully carried out the project entitled **"FAKEDTECT: AI-DRIVEN MULTIMODAL FAKE NEWS AND DEEPFAKE DETECTION SYSTEM"** during the academic year **2025 - 2026**.

Dr. V. Dhanalaxmi
Professor & Head

Internal Examiner

External Examiner

DECLARATION

I hereby declare that the project work presented in this thesis entitled **"FAKEDTECT: AI-DRIVEN MULTIMODAL FAKE NEWS AND DEEPFAKE DETECTION SYSTEM"** has been carried out by me in the **Department of Mathematics, University College of Science, Osmania University, Hyderabad**, as a part of the course curriculum for the **IV Semester of the Master of Science in Computer Science** degree.

I further declare that this work is my original work and has not been submitted, either in whole or in part, to any other University or Institution for the award of any other academic degree.

Date: 04-08-2026.

Ravalkol Pravalika.

H.T.No: 1007-24-504-011.

*M.Sc (Computer Science),
Department of Mathematics,
University College of Science,
Osmania University.*

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without acknowledging the people whose constant encouragement and support have been a source of inspiration throughout the completion of this project.

I express my sincere gratitude to **Prof. V. Dhanalaxmi, Professor and Head, Department of Mathematics**, for her encouragement and support extended during the course of this project.

I also extend my heartfelt thanks to the faculty members of the **Department of Mathematics, University College of Science, Osmania University**, for their support and cooperation during the completion of this project.

RAVALKOL PRAVALIKA

H.T. No.: 1007-24-504-011

M.Sc. Computer Science (IV Semester)

University College of Science

Osmania University, Hyderabad

ABSTRACT

The rapid growth of digital media has fueled misinformation through text-based fake news and AI-generated deepfakes. Existing detection systems are largely unimodal, limiting their effectiveness in scenarios where multiple forms of misinformation coexist. This project presents **FakeDetect**, an AI-driven multimodal detection system that integrates natural language processing and deep learning techniques to identify misinformation across text, image, and video.

A fine-tuned **BERT** model detects fake news with **99.13% accuracy**. An **EfficientNetB0** convolutional neural network, trained on more than **288,000 images** from three public datasets, detects deepfake images with **92.34% accuracy** and is extended to video analysis through frame-level classification, achieving **99.89% clip-level accuracy**. **Grad-CAM** provides visual explainability by highlighting the image regions that influence model predictions, while **Test-Time Augmentation (TTA)** improves the stability of image and video predictions.

The system is implemented using **PyTorch** and deployed through a **Streamlit** web application, enabling users to upload text, images, or videos and receive real-time predictions with confidence scores and visual explanations. Evaluated using **accuracy, precision, recall, and F1-score**, FakeDetect demonstrates that multimodal integration provides a more robust and reliable solution than single-modality approaches, with practical applications in media verification, social media monitoring, cybersecurity, and digital forensics.

TABLE OF CONTENTS

CONTENTS	PAGE NUMBERS
CHAPTER - 1 INTRODUCTION	1 - 7
1.1 Background	
1.2 Problem Statement	
1.3 Need for the Project	
1.4 Scope of the Project	
1.5 System Overview	
1.6 Applications	
CHAPTER - 2 OBJECTIVES	8
CHAPTER - 3 SYSTEM ANALYSIS	9 - 10
3.1 Existing System	
3.2 Proposed System	
CHAPTER - 4 REQUIREMENTS AND SPECIFICATIONS	11 - 12
4.1 System Requirements	
CHAPTER - 5 SYSTEM ARCHITECTURE	13 - 16
5.1 Architecture Overview	
CHAPTER - 6 IMPLEMENTATION	17 - 40
6.1 Overview	
6.2 Notebook 1: BERT Fake News Detection	
6.3 Notebook 2: Image Deepfake Detection using EfficientNetB0	

6.4 Notebook 3: Video Deepfake Detection

6.5 Notebook 4: Multimodal Fusion

6.6 Notebook 5: Streamlit Web Application

CHAPTER - 7 OUTPUT SNAPSHOTS

41 - 69

7.1 Fake News Detection Outputs

7.2 Deepfake Image Detection Outputs

7.3 Deepfake Video Detection Outputs

7.4 Combined CNN Outputs

7.5 Multimodal Fusion Outputs

7.6 Streamlit Web Application Snapshots

CHAPTER - 8 RESULTS AND DISCUSSION

70 - 74

8.1 Overview

8.2 Fake News Detection - BERT

8.3 Deepfake Image Detection - EfficientNetB0 V3

8.4 Deepfake Video Detection - CNN + OpenCV

8.5 Multimodal Fusion

8.6 Summary

CHAPTER - 9 CONCLUSION & FUTURE WORK

75 - 76

9.1 Conclusion

9.2 Future Work

CHAPTER - 10 BIBLIOGRAPHY

77 - 78

CHAPTER – 1

INTRODUCTION

1.1 Background

The widespread adoption of social media and digital communication technologies has fundamentally transformed the way information is created, shared, and consumed. While this has enabled rapid access to knowledge and news, it has simultaneously created fertile ground for the propagation of misinformation. Among the most concerning forms of digital misinformation are fake news and deepfake content. Fake news refers to deliberately fabricated or misleading textual information presented as factual reporting, often designed to influence public opinion, distort political discourse, or cause social harm. Deepfakes, on the other hand, are synthetically generated or manipulated images and videos produced using generative artificial intelligence techniques, capable of depicting events or statements that never occurred. The volume and sophistication of such content has grown considerably, making manual verification increasingly impractical and creating an urgent demand for automated detection solutions.



Figure 2.1 illustrates examples of real and manipulated images generated using modern artificial intelligence techniques. Although the manipulated versions appear visually convincing, they contain subtle alterations that are often difficult for humans to identify through manual inspection. As deepfake technology continues to advance, distinguishing authentic media from synthetic content has become increasingly challenging, highlighting the need for intelligent automated detection systems.

Figure 1.1: Examples of real and AI-manipulated images demonstrating different types of visual manipulation.

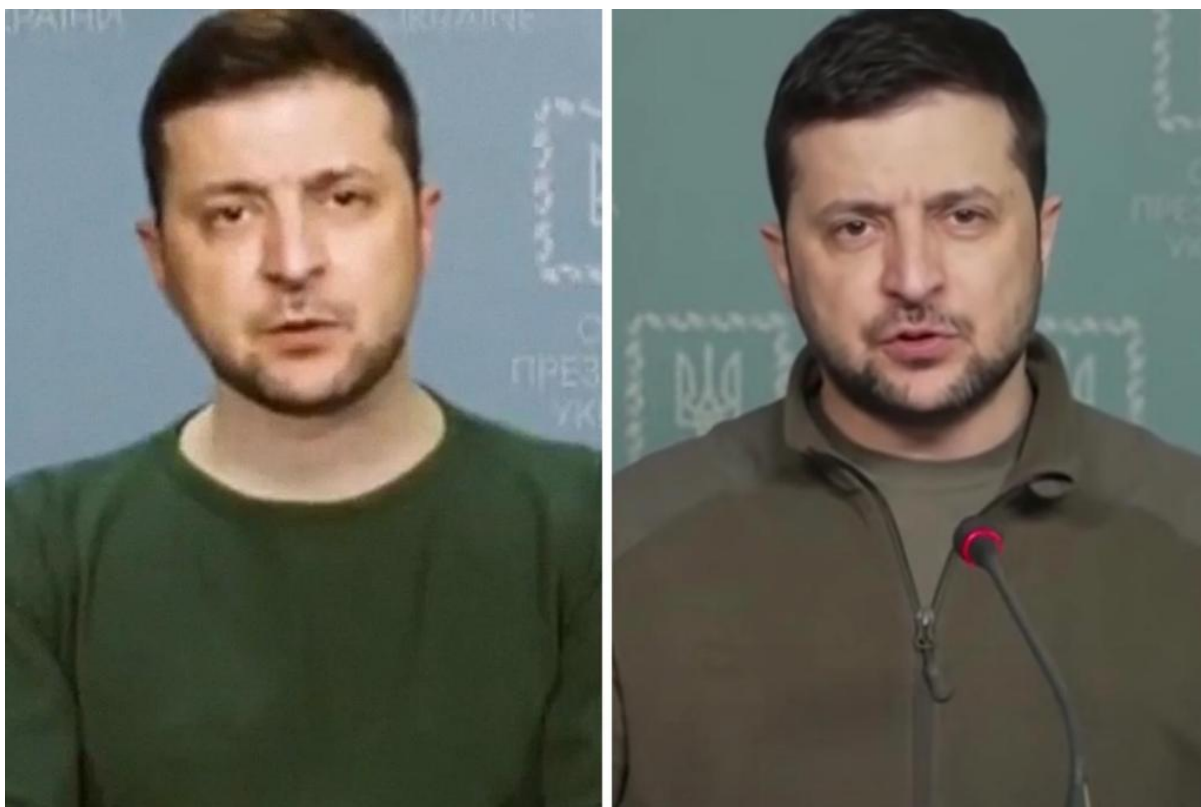


Figure 1.1(a): AI-generated deepfake image (left) versus the authentic image(right).

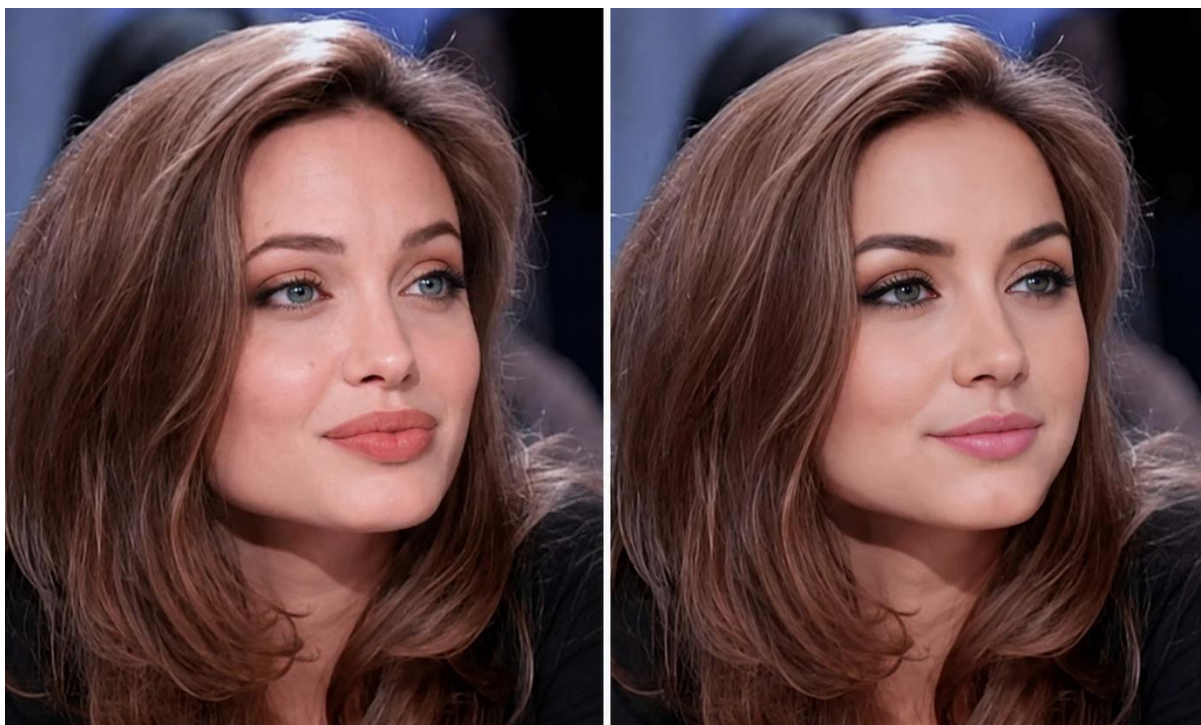


Figure 1.1(b): Original facial image (left) and AI-generated face-swapped image (right), demonstrating facial identity manipulation.



Figure 1.1(c): Original group photograph (left) and AI-generated face-swapped image (right), demonstrating identity manipulation using deepfake techniques.



Figure 1.1(d): Comparison of an original facial image (left) and its AI-enhanced beautified version (right), illustrating the visual alterations introduced by AI-based image enhancement.

1.2 Problem Statement

Current detection systems are predominantly designed around a single data modality - either analysing textual content for fake news or examining visual content for deepfake manipulation. Text-based detection systems often rely on shallow linguistic features or earlier-generation machine learning approaches that fail to capture the full semantic context of a message. Visual deepfake detectors, while more advanced in their use of convolutional architectures, are trained on specific manipulation techniques and can struggle to generalise across diverse generation methods. Critically, neither category of system is equipped to analyse both text and visual media simultaneously, despite the fact that real-world misinformation frequently combines both. A news article with a manipulated image, or a social media post pairing a misleading caption with a synthetic video, cannot be reliably assessed by a tool designed for only one format. This fragmentation represents a significant gap in the current state of misinformation detection.

1.3 Need for the Project

The coexistence of textual and visual misinformation in everyday digital content makes it essential to develop a system that can evaluate multiple data types within a unified framework. A multimodal approach allows the system to leverage complementary signals - for instance, detecting inconsistencies between a news headline and the accompanying image, or corroborating a suspicious text claim with evidence from an uploaded video. Such integration improves both detection accuracy and the reliability of the final verdict, reducing the risk of false positives and missed detections. Furthermore, users and fact-checkers increasingly require tools that are accessible, interpretable, and capable of handling diverse content formats without switching between separate platforms.

1.4 Scope of the Project

The scope of this project is to develop a multimodal artificial intelligence system capable of detecting misinformation across text, images, and videos. The system focuses on identifying fake news using a fine-tuned BERT model and detecting manipulated visual content using the EfficientNetB0 convolutional neural network. Predictions from the individual models are combined through a weighted multimodal fusion mechanism to provide a unified classification result. The system also incorporates Grad-CAM to improve model interpretability by highlighting the image regions that influence predictions. Finally, the complete solution is deployed as a Streamlit web application, enabling users to perform real-time fake content detection through an intuitive interface.

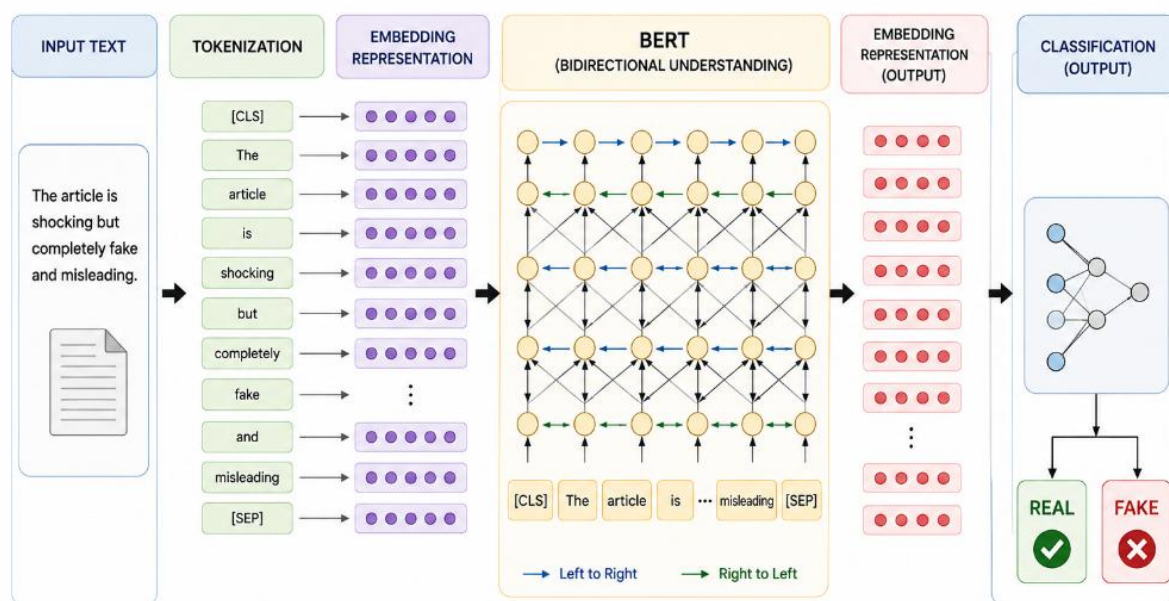
1.5 System Overview

FakeDetect is a multimodal artificial intelligence system designed to detect fake news and deepfake content from text, images, and videos. The system consists of five major

components that work together to analyse different types of digital content and generate a unified prediction.

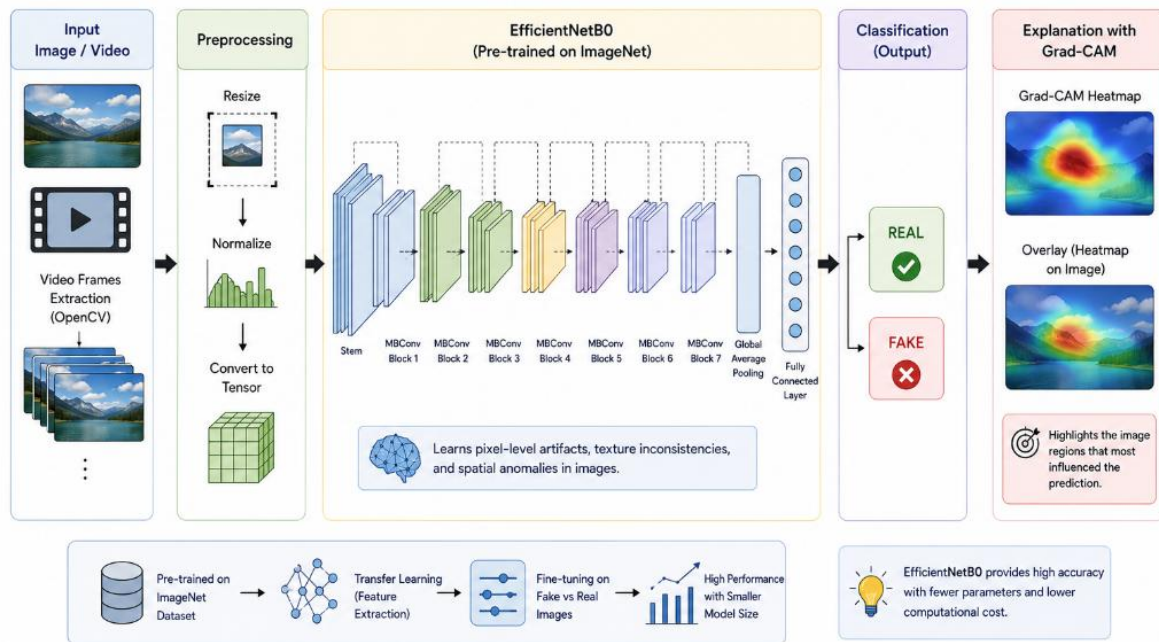
BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based language model pre-trained on large corpora of text. By reading text bidirectionally - considering both left and right context for every word simultaneously - BERT develops a rich understanding of semantic relationships and contextual meaning. In this system, BERT is fine-tuned on a labelled dataset of genuine and fabricated news articles to classify text inputs as either real or fake with high precision.

BERT WORKFLOW



EfficientNetB0 is a compact yet high-performing convolutional neural network architecture pre-trained on the ImageNet dataset. It serves as the visual backbone of FakeDetect, processing both still images and individual video frames extracted by OpenCV. The model learns to identify pixel-level artifacts, texture inconsistencies, and spatial anomalies characteristic of AI-generated or manipulated visual content. Gradient-weighted Class Activation Mapping (Grad-CAM) is applied to highlight the image regions that most influenced each classification, providing a visual explanation for the model's verdict.

EfficientNetB0 Workflow



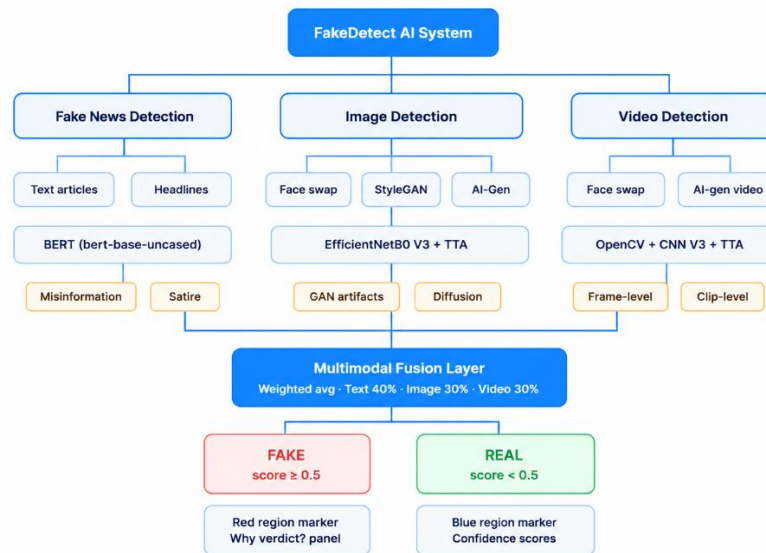
OpenCV is used for video preprocessing and analysis. Since convolutional neural networks operate on images rather than videos directly, OpenCV extracts evenly spaced frames from uploaded videos before they are processed by EfficientNetB0. The prediction probabilities obtained from all analysed frames are averaged to determine the final classification of the video, enabling reliable detection of manipulated video content.

Grad-CAM (Gradient-weighted Class Activation Mapping) provides visual explainability for image predictions. It generates heatmaps that highlight the regions of an image that contributed most to the model's decision. This improves the transparency and interpretability of the deep learning model by allowing users to understand why a particular image was classified as real or fake.

Multimodal Fusion integrates the predictions generated by the text, image, and video models into a single final decision. A weighted average strategy is employed, assigning 40% weight to the BERT text model and 30% each to the image and video detection models. When fewer than three modalities are available, the weights are automatically normalised to ensure fair contribution from the available inputs. This approach improves the robustness and reliability of the overall prediction while allowing the system to process any combination of text, image, and video inputs.

Streamlit is used as the deployment framework to provide an interactive web-based interface for the FakeDetect system. Through this interface, users can enter news text, upload images, or submit videos for analysis. The application loads the trained models, performs real-time inference, and displays prediction results along with confidence scores. For image analysis, Grad-CAM visualisations are also presented to highlight the regions influencing the model's decision. The web application enables users to access the system easily without requiring

technical expertise, making FakeDetect suitable for practical applications such as social media monitoring, journalism, cybersecurity, and digital forensics.



1.6 Applications

The FakeDetect system has broad applicability across several domains where the verification of digital content is critical.

- **Social Media Monitoring:** Helps detect fake news and deepfake content before it spreads across social media platforms.
- **News Verification and Journalism:** Assists journalists and fact-checkers in verifying the authenticity of news articles, images, and videos.
- **Cybersecurity:** Helps identify phishing campaigns, identity fraud, and misinformation using AI-generated content.
- **Digital Forensics:** Supports the verification of images and videos during digital investigations and legal processes.
- **Public Awareness Platforms:** Helps users identify AI-generated misinformation and verify digital content more effectively.

CHAPTER – 2

OBJECTIVES

The objectives of the proposed **FakeDetect** system are:

- To develop an AI-driven multimodal system capable of detecting fake news and deepfake content from text, images, and videos.
- To classify news articles as real or fake using a fine-tuned BERT model.
- To detect manipulated and AI-generated images and videos using the EfficientNetB0 convolutional neural network.
- To preprocess textual, image, and video data for effective model training and inference.
- To integrate predictions from multiple modalities using a multimodal fusion strategy for improved detection performance.
- To provide visual explainability for image predictions using Gradient-weighted Class Activation Mapping (Grad-CAM).
- To evaluate the performance of the proposed models using accuracy, precision, recall, F1-score, confusion matrix, and classification report.
- To deploy the proposed system as an interactive Streamlit web application for real-time fake content detection.
- To demonstrate the applicability of the system in social media monitoring, news verification, cybersecurity, and digital forensics.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 Existing System

Existing fake news and deepfake detection systems are generally designed to analyze only one type of data at a time. Fake news detection systems mainly focus on textual content, while deepfake detection systems analyze images or videos separately. Since misinformation on digital platforms often contains both text and visual media, these systems are not sufficient for real-world scenarios.

- Most fake news detection systems use traditional Natural Language Processing (NLP) or basic machine learning techniques to classify news articles. These methods often fail to understand the complete context and meaning of the text, resulting in lower accuracy for complex misinformation.
- Deepfake detection systems use Convolutional Neural Networks (CNNs) to identify manipulated images and videos by detecting visual inconsistencies and pixel-level artifacts. However, highly realistic AI-generated media remains difficult to detect.
- Most existing systems operate independently for text and visual data, requiring users to use different tools for different types of content.

3.1.1 Limitations of Existing System

- Supports only a single data type (text or image/video).
- Limited understanding of contextual information in textual content.
- Lower accuracy for advanced AI-generated deepfakes.
- Requires separate tools for text and visual content.
- Does not provide a unified solution for detecting multimodal misinformation.
- Limited explainability of model predictions.

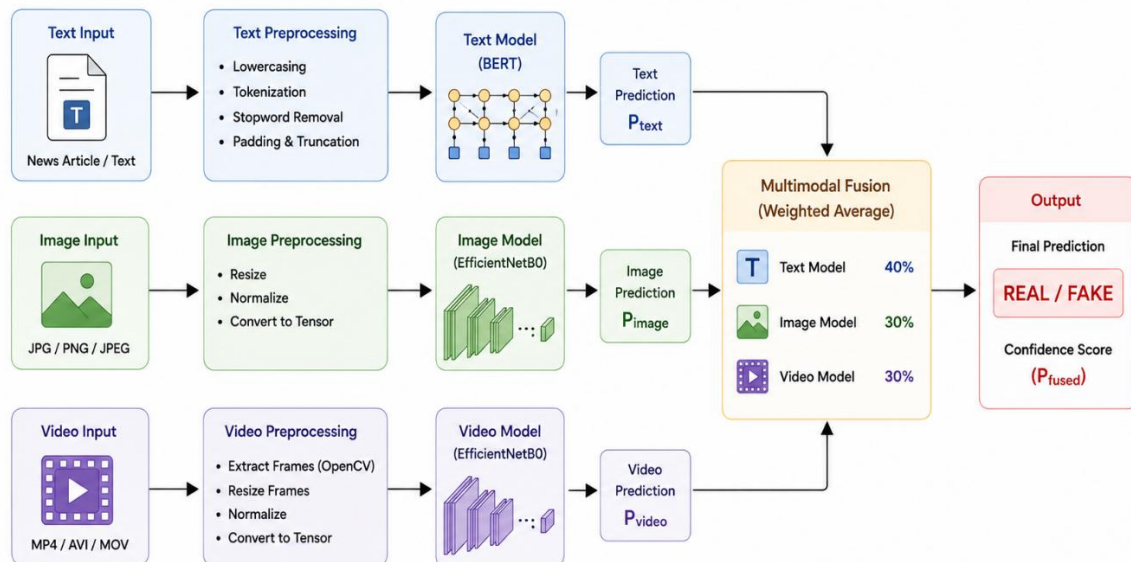
3.2 Proposed System

The proposed system, **FakeDetect**, is an AI-driven multimodal fake news and deepfake detection system that analyzes **text, images, and videos** within a single platform. It combines Natural Language Processing and Deep Learning techniques to improve the accuracy and reliability of misinformation detection.

The system uses **BERT** to classify textual news as real or fake by understanding the contextual meaning of the text. **EfficientNetB0**, a CNN architecture, is used to detect manipulated images and video frames extracted using OpenCV. The predictions from the text, image, and video models are combined using a **multimodal fusion** technique to generate a final prediction with a confidence score. **Grad-CAM** provides visual explanations by

highlighting the image regions that influenced the prediction. The entire system is deployed through a **Streamlit** web application, allowing users to upload text, images, or videos and receive real-time results.

Proposed System Architecture



3.2.1 Advantages of the Proposed System

- Supports **multimodal detection** by analyzing text, images, and videos together.
- Uses **BERT** for better contextual understanding of textual content.
- Uses **EfficientNetB0** to accurately detect manipulated images and video frames.
- Combines predictions using **multimodal fusion** for improved overall accuracy.
- Provides **Grad-CAM visualizations** to explain image classification results.
- Offers a **single integrated platform** instead of separate tools.
- Generates **real-time predictions** through a user-friendly Streamlit interface.
- Provides **confidence scores** along with prediction results.
- Suitable for applications such as **social media monitoring, news verification, cybersecurity, and digital forensics**.

CHAPTER – 4

REQUIREMENTS AND SPECIFICATIONS

4.1 System Requirements

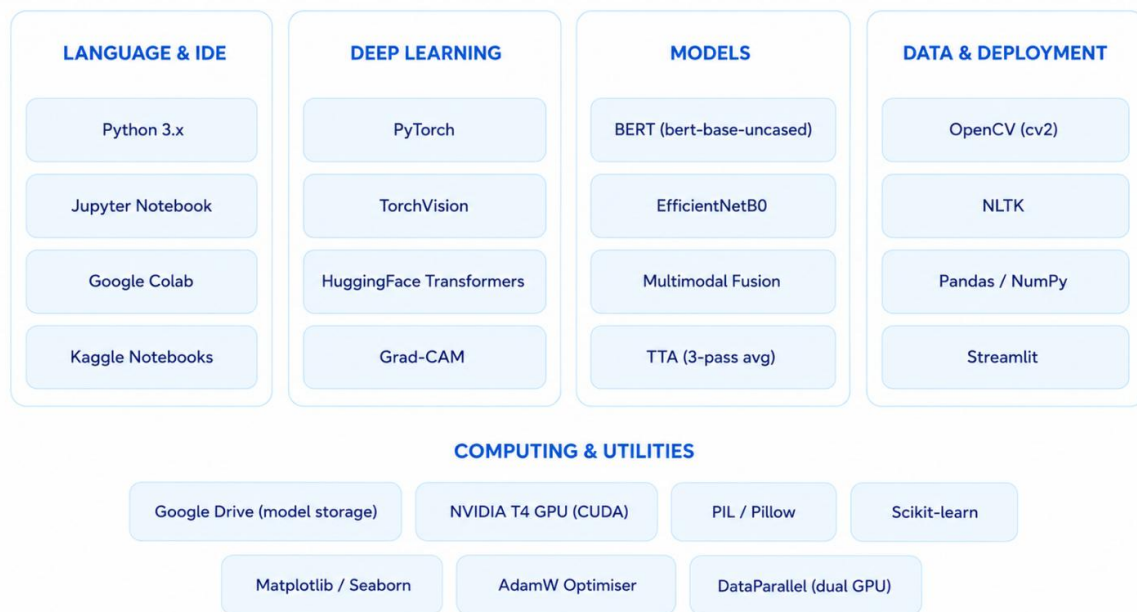
4.1.1 Hardware Requirements

- **Processor:** Intel Core i5/i7 (11th Gen or above) or AMD Ryzen 5/7 equivalent
- **GPU:** NVIDIA GPU with CUDA support (e.g., GTX 1650 / RTX 3050 or above) - optional but recommended
- **RAM:** Minimum 8 GB (16 GB recommended)
- **Storage:** Minimum 256 GB SSD (512 GB SSD recommended)
- **System Type:** 64-bit Operating System, x64-based processor

4.1.2 Software Requirements

- **Operating System:** Windows 10/11 (64-bit) or Ubuntu 20.04+
- **Programming Language:** Python 3.8 or above
- **IDE / Environment:** Jupyter Notebook, Visual Studio Code, or Google Colab
- **Deep Learning Framework:** PyTorch
- **CNN Architecture:** TorchVision (EfficientNetB0)
- **NLP Library:** HuggingFace Transformers (bert-base-uncased), NLTK
- **Image and Video Processing:** OpenCV (cv2), PIL / Pillow
- **Data Handling:** Pandas, NumPy
- **Machine Learning Library:** Scikit-learn
- **Explainability:** pytorch-grad-cam (Grad-CAM)
- **Visualisation:** Matplotlib, Seaborn
- **Frontend / Deployment:** Streamlit
- **Cloud Storage:** Google Drive

Figure 4.1: Tools and Technologies Used



4.1.3 Dataset Requirements

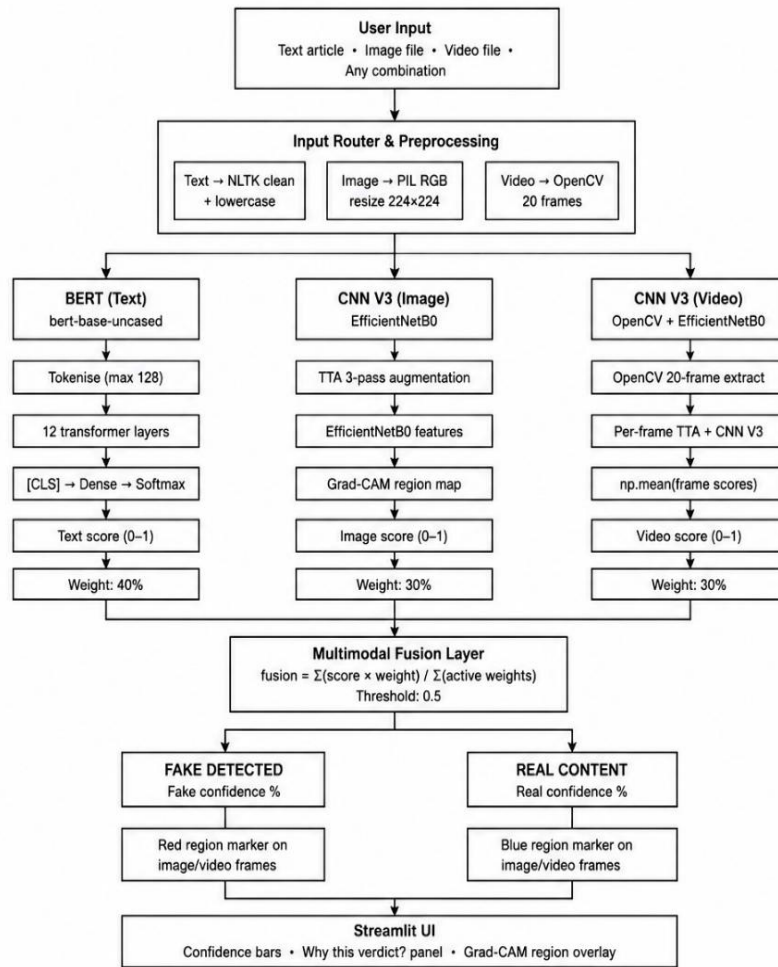
- **Fake and Real News Dataset** - 44,898 labelled news articles (Fake.csv + True.csv) - used for BERT training
- **LIAR Dataset** - 12,836 political statements (train.tsv, valid.tsv, test.tsv) - used as secondary text dataset
- **Deepfake vs Real 60K** - 190,335 face images - used for CNN image training
- **140K Real and Fake Faces** - 140,000 StyleGAN and real face images - used to improve AI-generated face detection
- **AI-Generated vs Real** - 48,000 diverse AI-generated and real images - used to extend CNN generalisation
- **Real/Fake Video Dataset** - 872 video clips - used for video deepfake detection fine-tuning

CHAPTER – 5

SYSTEM ARCHITECTURE

5.1 Architecture Overview

The FakeDetect system follows a multimodal architecture that analyzes textual, image, and video inputs within a single framework. Each input is processed through its respective preprocessing stage before being analyzed by the appropriate AI model. The predictions generated by the individual models are combined using a multimodal fusion mechanism to produce a final classification. The result is then displayed to the user through a Streamlit-based web interface along with the confidence score and visual explanation for image and video predictions.

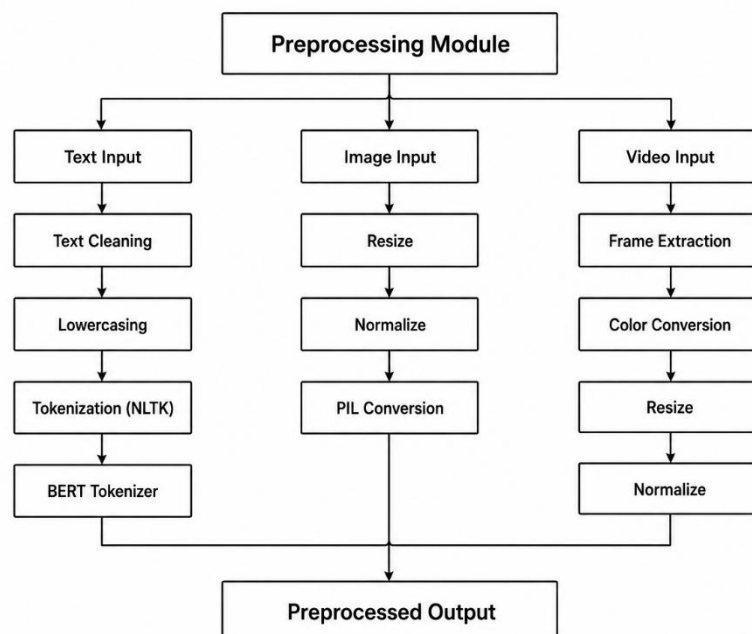


5.1.1 Input Layer

The system accepts three different types of user inputs: text, images, and videos. Users can either enter textual content or upload image and video files through the Streamlit interface. This flexible input mechanism allows the system to detect misinformation from different forms of digital content.

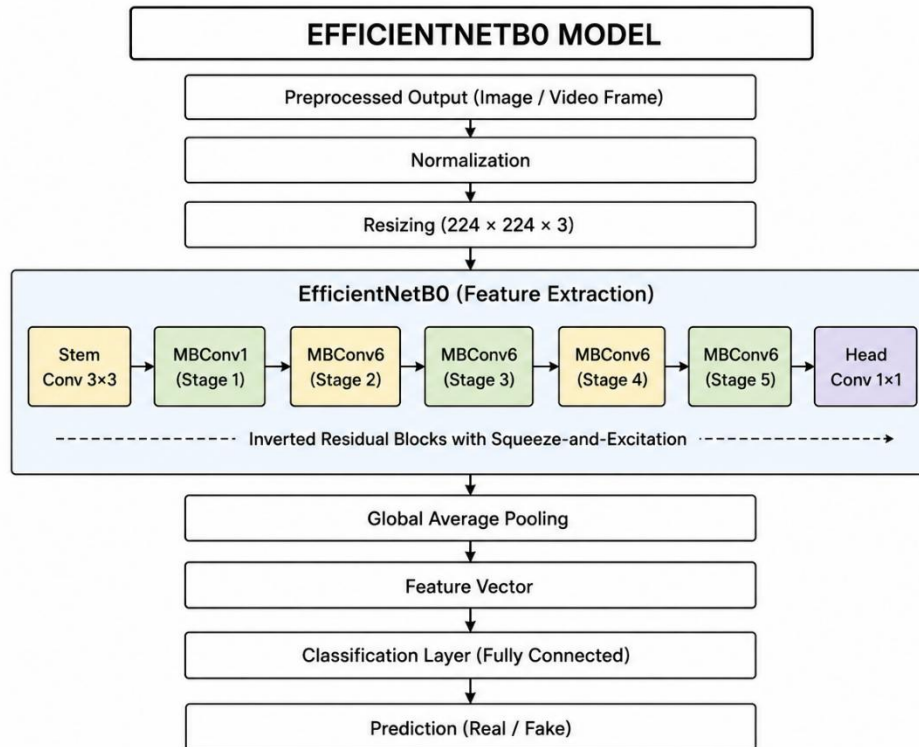
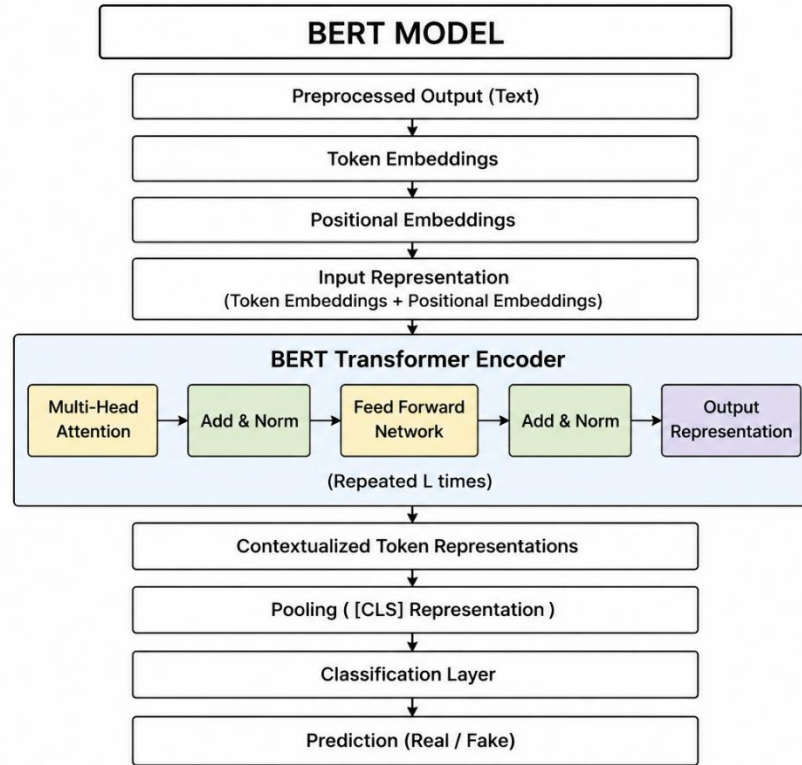
5.1.2 Preprocessing Layer

Before analysis, each input undergoes preprocessing to improve the quality and consistency of the data. Text is cleaned and tokenized, images are resized and normalized, and videos are divided into individual frames. These preprocessing steps ensure that the data is suitable for the AI models used in the system.



5.1.3 Detection Models

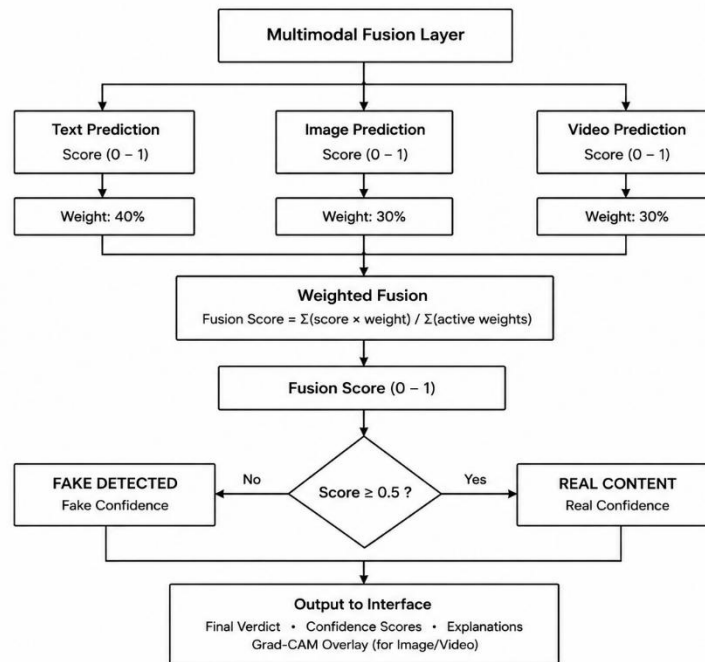
The processed data is analyzed using dedicated AI models. The BERT model is responsible for detecting fake news from textual content by understanding the contextual meaning of the text. EfficientNetB0 is used to analyze images and video frames to identify manipulated or AI-generated visual content. Each model independently generates a prediction based on its respective input.



5.1.4 Multimodal Fusion Layer

The predictions obtained from the text, image, and video models are combined using a multimodal fusion mechanism. This layer integrates the outputs of all available modalities and produces a single final prediction. By combining information from multiple sources, the

system improves detection accuracy and provides more reliable results than single-modality approaches.



5.1.5 Explainability Layer

To improve transparency, the system provides visual explanations for image and video predictions using Grad-CAM. The highlighted regions indicate the areas that contributed most to the model's decision, helping users better understand the classification results.

5.1.6 Output Layer

The final output displays whether the submitted content is classified as **Real** or **Fake**. Along with the prediction, the system presents a confidence score. For image and video inputs, the corresponding Grad-CAM visualization is also displayed to explain the model's decision.

5.1.7 Deployment Layer

The complete system is deployed using Streamlit, providing a simple and interactive web interface. The trained models are loaded from Google Drive, and the application is executed through Google Colab using port forwarding. This setup allows users to interact with the system and receive predictions in real time during deployment.

CHAPTER – 6

IMPLEMENTATION

6.1 Overview

This chapter describes the implementation of the proposed FakeDetect system for detecting fake news and deepfake content using artificial intelligence. The implementation was carried out using Python in the Google Colab environment, with separate notebooks developed for text classification, image deepfake detection, video deepfake detection, multimodal fusion, and web application deployment.

The text detection module was implemented using a fine-tuned BERT model to classify news articles as real or fake. The image and video detection modules were developed using the EfficientNetB0 convolutional neural network, where video analysis was performed by extracting representative frames using OpenCV and classifying each frame individually. A combined CNN training notebook was also developed to improve the model's ability to detect different types of AI-generated and manipulated images using multiple publicly available datasets.

To provide comprehensive detection, a multimodal fusion module was implemented to combine predictions from text, image, and video inputs and generate a unified confidence score and final verdict. Finally, the trained models were integrated into a Streamlit-based web application, enabling users to upload content and obtain real-time predictions through an interactive interface.

The following sections describe the implementation process of each module, including data preprocessing, model development, training, evaluation, multimodal fusion, and deployment.

6.2 Notebook 1: BERT Fake News Detection

6.2.1 Loading Required Libraries and Dataset

```
import pandas as pd

import numpy as np

import torch

from transformers import BertTokenizer, BertForSequenceClassification

from transformers import get_linear_schedule_with_warmup

from torch.optim import AdamW

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
classification_report

from nltk.corpus import stopwords

import re
```

```

import string
import warnings
warnings.filterwarnings("ignore")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

6.2.2 Loading the Fake and Real News Dataset

```

fake_df = pd.read_csv(FAKE_CSV)
true_df = pd.read_csv(TRUE_CSV)

```

6.2.3 Data Labeling and Dataset Preparation

```

fake_df["label"] = 1
true_df["label"] = 0

fake_df["content"] = fake_df["title"].fillna("") + " " + fake_df["text"].fillna("")
true_df["content"] = true_df["title"].fillna("") + " " + true_df["text"].fillna("")

df = pd.concat(
    [fake_df[["content", "label"]], true_df[["content", "label"]]], ignore_index=True)

df = df.sample(frac=1, random_state=42).reset_index(drop=True)
df = df[df["content"].str.strip() != ""].reset_index(drop=True)

```

6.2.4 Text Preprocessing

```

stop_words = set(stopwords.words("english"))

def clean_text(text):
    if not isinstance(text, str):
        return ""

    text = text.lower()

```

```

text = re.sub(r'http\S+|www\S+|https\S+', '', text)
text = re.sub(r'<.*?>', '', text)
text = text.translate(str.maketrans("", "", string.punctuation))
text = re.sub(r'\d+', '', text)

tokens = [word for word in text.split() if word not in stop_words and len(word) > 2]
return " ".join(tokens)

```

```

df["cleaned_content"] = df["content"].apply(clean_text)
df = df[df["cleaned_content"].str.strip() != ""].reset_index(drop=True)

```

6.2.5 BERT Tokenization

```

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

tokens = tokenizer(X_train[0], max_length=MAX_LEN, truncation=True,
padding="max_length", return_tensors="pt")

```

6.2.6 Loading the Pre-trained BERT Model

```

model = BertForSequenceClassification.from_pretrained('bert-base-uncased',
num_labels=2 )

model = model.to(device)

```

6.2.7 Model Configuration

```

EPOCHS = 3

LEARNING_RATE = 2e-5

WARMUP_STEPS = 0

optimizer = AdamW(model.parameters(), lr=LEARNING_RATE, eps=1e-8)

scheduler = get_linear_schedule_with_warmup(optimizer,
num_warmup_steps=WARMUP_STEPS, num_training_steps=len(train_loader) *
EPOCHS)

```

6.2.8 Model Training

```
def train_epoch(model, loader, optimizer, scheduler, device):  
    model.train()  
    total_loss, correct, total = 0, 0, 0  
  
    for batch in loader:  
        input_ids = batch["input_ids"].to(device)  
        attention_mask = batch["attention_mask"].to(device)  
        labels = batch["label"].to(device)  
  
        optimizer.zero_grad()  
        outputs = model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)  
        loss = outputs.loss  
        loss.backward()  
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)  
        optimizer.step()  
        scheduler.step()  
  
        preds = torch.argmax(outputs.logits, dim=1)  
        total_loss += loss.item()  
        correct += (preds == labels).sum().item()  
        total += labels.size(0)  
  
    return total_loss / len(loader), correct / total  
  
def eval_epoch(model, loader, device):  
    model.eval()  
    total_loss, correct, total = 0, 0, 0  
    all_preds, all_labels = [], []
```

```

with torch.no_grad():
    for batch in loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device)

        outputs = model(input_ids=input_ids, attention_mask=attention_mask,
labels=labels)

        loss = outputs.loss
        preds = torch.argmax(outputs.logits, dim=1)
        total_loss += loss.item()
        correct += (preds == labels).sum().item()
        total += labels.size(0)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

    return total_loss / len(loader), correct / total, all_preds, all_labels

history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}

for epoch in range(EPOCHS):
    train_loss, train_acc = train_epoch(model, train_loader, optimizer, scheduler, device)
    val_loss, val_acc, val_preds, val_labels = eval_epoch(model, val_loader, device)
    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["train_acc"].append(train_acc)
    history["val_acc"].append(val_acc)

```

6.2.9 Model Evaluation

```

test_loss, test_acc, test_preds, test_labels = eval_epoch(model, test_loader, device)

```

```

accuracy = accuracy_score(test_labels, test_preds)
precision = precision_score(test_labels, test_preds)
recall   = recall_score(test_labels, test_preds)
f1       = f1_score(test_labels, test_preds)

```

6.2.10 Saving the Trained Model

```

model.save_pretrained(MODEL_SAVE_PATH)
tokenizer.save_pretrained(MODEL_SAVE_PATH)

```

6.2.11 Fake News Prediction Function

```

def predict_news(text, model, tokenizer, device, max_len=128):
    model.eval()

    cleaned = clean_text(text)

    encoding = tokenizer( cleaned, max_length=max_len, truncation=True,
padding="max_length", return_tensors="pt" )

    with torch.no_grad():
        outputs = model(input_ids=encoding["input_ids"].to(device),
            attention_mask=encoding["attention_mask"].to(device) )

    probs = torch.softmax(outputs.logits, dim=1)
    pred = torch.argmax(probs, dim=1).item()

    label = "FAKE" if pred == 1 else "REAL"
    confidence = probs[0][pred].item() * 100

    return label, confidence

```

6.3 Notebook 2: Image Deepfake Detection using EfficientNetB0

6.3.1 Image Preprocessing

```
IMG_SIZE = 224
```

```
BATCH_SIZE = 32
```

```
train_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(0.5),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

```
test_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

```
train_dataset = datasets.ImageFolder(TRAIN_DIR, transform=train_transforms)
```

```
val_dataset = datasets.ImageFolder(VAL_DIR, transform=test_transforms)
```

```
test_dataset = datasets.ImageFolder(TEST_DIR, transform=test_transforms)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
```

```
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE)
```

```
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE)
```

6.3.2 EfficientNetB0 Model Construction

```
def build_efficientnet(num_classes=2, dropout=0.3):
```

```
    model = models.efficientnet_b0(weights="IMAGENET1K_V1")
```

```
    for param in model.parameters():
```

```

    param.requires_grad = False

    for param in model.features[6:].parameters():
        param.requires_grad = True

    in_features = model.classifier[1].in_features

    model.classifier = nn.Sequential(nn.Dropout(dropout), nn.Linear(in_features, 512),
nn.ReLU(), nn.Dropout(dropout / 2), nn.Linear(512, num_classes) )

    return model

model = build_efficientnet().to(device)

```

6.3.3 Model Training Configuration

```

EPOCHS = 10

LEARNING_RATE = 1e-4

criterion = nn.CrossEntropyLoss()

optimizer = optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()),
    lr=LEARNING_RATE, weight_decay=1e-4 )

scheduler =
optim.lr_scheduler.ReduceLROnPlateau(optimizer,mode="min",factor=0.3,patience=2)

```

6.3.4 Model Training

```

def train_epoch(model, loader, criterion, optimizer, device):
    model.train()

    total_loss, correct, total = 0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

```

```

optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
preds = torch.argmax(outputs, dim=1)
total_loss += loss.item()
correct += (preds == labels).sum().item()
total += labels.size(0)

return total_loss / len(loader), correct / total

def eval_epoch(model, loader, criterion, device):
    model.eval()
    total_loss, correct, total = 0, 0, 0
    all_preds, all_labels = [], []

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)
            preds = torch.argmax(outputs, dim=1)

            total_loss += loss.item()
            correct += (preds == labels).sum().item()
            total += labels.size(0)

```

```

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

    return total_loss / len(loader), correct / total, all_preds, all_labels

history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
best_val_acc = 0

for epoch in range(EPOCHS):
    train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer, device)
    val_loss, val_acc, val_preds, val_labels = eval_epoch(model, val_loader, criterion, device)

    scheduler.step(val_loss)
    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["train_acc"].append(train_acc)
    history["val_acc"].append(val_acc)

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(),
                    f'{MODEL_SAVE_PATH}/efficientnet_best.pth')

```

6.3.5 Model Evaluation

```

_, test_acc, test_preds, test_labels = eval_epoch(
    model, test_loader, criterion, device)

acc = accuracy_score(test_labels, test_preds)
prec = precision_score(test_labels, test_preds)
rec = recall_score(test_labels, test_preds)

```

```
f1 = f1_score(test_labels, test_preds)
print(classification_report(test_labels, test_preds, target_names=["Fake", "Real"]))
```

6.3.6 Saving the Trained Model

```
torch.save(model.state_dict(), f'{MODEL_SAVE_PATH}/efficientnet_final.pth')
```

6.3.7 Image Prediction Function

```
from PIL import Image

def predict_image(image_path, model, device, img_size=224):
    model.eval()

    transform = transforms.Compose([
        transforms.Resize((img_size, img_size)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

    image = Image.open(image_path).convert("RGB")
    tensor = transform(image).unsqueeze(0).to(device)

    with torch.no_grad():
        probs = torch.softmax(model(tensor), dim=1)

    pred = torch.argmax(probs, dim=1).item()
    label = "DEEPFAKE" if pred == 0 else "REAL"

    return (label, probs[0][1].item() * 100, probs[0][0].item() * 100)
```

6.3.8 Test-Time Augmentation (TTA)

```
def eval_with_tta(model, device):
    model.eval()

    all_preds, all_labels = [], []
```

```

tta_transforms = [transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406], [0.229,0.224,0.225])
]),
transforms.Compose([
    transforms.Resize((224,224)),
    transforms.RandomHorizontalFlip(1.0),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406], [0.229,0.224,0.225])
]),
transforms.Compose([
    transforms.Resize((256,256)),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406], [0.229,0.224,0.225])
])]

test_data = datasets.ImageFolder(TEST_DIR)
with torch.no_grad():
    for img_path, label in test_data.samples:
        img = Image.open(img_path).convert("RGB")

        probs = torch.zeros(2).to(device)

        for tfm in tta_transforms:
            tensor = tfm(img).unsqueeze(0).to(device)
            probs += torch.softmax(model(tensor), dim=1).squeeze(0)

        pred = torch.argmax(probs / len(tta_transforms)).item()

```

```

        all_preds.append(pred)
        all_labels.append(label)
    return all_preds, all_labels

```

6.3.9 GRAD-CAM

```

from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image

cam = GradCAM(model=model, target_layers=[model.features[-1]])

val_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize( [0.485,0.456,0.406], [0.229,0.224,0.225] ) ])

def predict_with_gradcam(image_path):
    image = Image.open(image_path).convert("RGB")
    resized = image.resize((224,224))
    rgb = np.array(resized).astype(np.float32) / 255.0

    tensor = val_transform(image).unsqueeze(0).to(device)

    with torch.no_grad():
        probs = torch.softmax(model(tensor), dim=1)

    heatmap = cam(input_tensor=tensor, targets=None)[0]
    overlay = show_cam_on_image(rgb, heatmap, use_rgb=True)

    return (overlay, probs[0][1].item() * 100, probs[0][0].item() * 100)

```

6.4 Notebook 3: Video Deepfake Detection

6.4.1 Loading the Trained EfficientNetB0 Model

```
model = build_efficientnet()
weights = torch.load(CNN_MODEL_PATH, map_location=device)

model.load_state_dict(weights['model_state'] if 'model_state' in weights else weights)

model = model.to(device)
model.eval()
```

6.4.2 Frame Preprocessing and Clip Prediction Function

```
frame_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225] )
])

def predict_clip(folder_path, model, device):
    frame_files = sorted([
        f for f in os.listdir(folder_path)
        if f.endswith((".jpg", ".jpeg", ".png"))
    ])
    fake_probs = []

    with torch.no_grad():
        for file in frame_files:
            img = Image.open(os.path.join(folder_path, file) ).convert("RGB")
            tensor = frame_transform(img).unsqueeze(0).to(device)
            probs = torch.softmax(model(tensor), dim=1)
            fake_probs.append(probs[0][0].item())
```

```
avg_fake = np.mean(fake_probs)
```

```
return ("DEEPFAKE" if avg_fake>=0.5 else "REAL", (1-avg_fake)*100,avg_fake*100)
```

6.4.3 Video Processing and Final Prediction

```
def predict_video(video_path, model, device, num_frames=20):
```

```
    cap = cv2.VideoCapture(video_path)
```

```
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

```
    indices = np.linspace(0, total-1, min(num_frames,total), dtype=int )
```

```
    frames = []
```

```
    fake_probs = []
```

```
    with torch.no_grad():
```

```
        for idx in indices:
```

```
            cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
```

```
            ret, frame = cap.read()
```

```
            if ret:
```

```
                frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
                img = Image.fromarray(frame)
```

```
                frames.append(img)
```

```
                tensor = frame_transform(img).unsqueeze(0).to(device)
```

```
                probs = torch.softmax(model(tensor),dim=1)
```

```
                fake_probs.append(probs[0][0].item())
```

```
    cap.release()
```

```
    avg_fake = np.mean(fake_probs)
```

```
    return (frames, (1-avg_fake)*100, avg_fake*100, "DEEPFAKE" if avg_fake>=0.5 else  
"REAL")
```

6.5 Notebook 4: Combined CNN Model Training

6.5.1 Loading and combining image datasets

```
IMG_SIZE = 224
BATCH_SIZE = 32

train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406], [0.229,0.224,0.225])
])

val_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406], [0.229,0.224,0.225])
])

deepfake_train = datasets.ImageFolder(DEEPFAKE_DIR,transform=train_transform)

cifake_train = datasets.ImageFolder(CIFAKE_TRAIN_DIR,transform=train_transform)

combined_train = ConcatDataset([deepfake_train, cifake_train])

train_loader = DataLoader(combined_train, batch_size=BATCH_SIZE, shuffle=True)
```

6.5.2 Image preprocessing

```
train_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2 ),
    transforms.ToTensor(),
```

```

        transforms.Normalize( [0.485,0.456,0.406], [0.229,0.224,0.225])
    ])

class FolderDataset(Dataset):
    def __init__(self, fake_dir, real_dir, transform):
        self.transform = transform

        fake = [(os.path.join(fake_dir,f),0) for f in os.listdir(fake_dir)]
        real = [ (os.path.join(real_dir,f),1)for f in os.listdir(real_dir)]

        self.samples = fake + real

        random.shuffle(self.samples)

    def __len__(self):
        return len(self.samples)

    def __getitem__(self,index):
        path,label = self.samples[index]

        image = Image.open(path).convert("RGB")

        return self.transform(image), label

```

6.5.3 EfficientNetB0 model architecture

```

model = build_efficientnet(num_classes=2, dropout=0.3)
weights = torch.load(ORIGINAL_MODEL, map_location=device)
model.load_state_dict(weights['model_state'] if 'model_state' in weights else weights)
model = model.to(device)

```

6.5.4 Training configuration

```

EPOCHS = 10

LEARNING_RATE = 5e-5

criterion = nn.CrossEntropyLoss()

optimizer = optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()),

```

```
lr=LEARNING_RATE, weight_decay=1e-4)
```

```
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=0.3,  
patience=2)
```

6.5.5 Training Loop

```
def train_epoch(model, loader, criterion, optimizer, device):
```

```
    model.train()
```

```
    total_loss, correct, total = 0, 0, 0
```

```
    for images, labels in loader:
```

```
        images, labels = images.to(device), labels.to(device)
```

```
        optimizer.zero_grad()
```

```
        outputs = model(images)
```

```
        loss = criterion(outputs, labels)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
        total_loss += loss.item()
```

```
        correct += (outputs.argmax(1) == labels).sum().item()
```

```
        total += labels.size(0)
```

```
    return total_loss / len(loader), correct / total
```

```
def eval_epoch(model, loader, device):
```

```
    model.eval()
```

```
    preds, labels_all = [], []
```

```
    with torch.no_grad():
```

```
        for images, labels in loader:
```

```
            images = images.to(device)
```

```
            outputs = model(images)
```

```
            preds.extend(outputs.argmax(1).cpu().numpy())
```

```

        labels_all.extend(labels.numpy())
    return preds, labels_all

best_acc = 0

for epoch in range(EPOCHS):
    train_epoch(model, train_loader, criterion, optimizer, device)
    preds, labels = eval_epoch(model, val_loader, device )
    acc = accuracy_score(labels, preds)
    if acc > best_acc:
        best_acc = acc
        torch.save(model.state_dict(), "efficientnet_v2_best.pth")

```

6.5.6 Model evaluation

```

_, _, test_preds, test_labels = eval_epoch(model, test_loader, device)

acc = accuracy_score(test_labels, test_preds)
prec = precision_score(test_labels, test_preds)
rec = recall_score(test_labels, test_preds)
f1 = f1_score(test_labels, test_preds)
print(classification_report(test_labels, test_preds, target_names=["Fake", "Real"]))

cm = confusion_matrix(test_labels, test_preds)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Fake", "Real"],
            yticklabels=["Fake", "Real"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

6.5.7 Saving the Trained Model

```

torch.save(model.state_dict(), f"{MODEL_SAVE_PATH}/efficientnet_v2_final.pth")

```

6.6 Notebook 5: Multimodal Fusion

6.6.1 Multimodal Fusion function

```
def multimodal_fusion(
    text=None,
    image_path=None,
    video_path=None,
    text_weight=0.4,
    image_weight=0.3,
    video_weight=0.3,
    threshold=0.5
):
    scores, weights = {}, {}

    if text:
        scores["text"] = predict_text(text, bert_model, bert_tokenizer, device)
        weights["text"] = text_weight

    if image_path:
        scores["image"] = predict_image(image_path, cnn_image_model, device)
        weights["image"] = image_weight

    if video_path:
        score = predict_video(video_path, cnn_video_model, device)
        if score is not None:
            scores["video"] = score
            weights["video"] = video_weight

    total = sum(weights.values())
    weights = {k: v / total for k, v in weights.items()}
```

```

fusion_score = sum(
    scores[k] * weights[k]
    for k in scores
)
return {
    "verdict": "FAKE" if fusion_score >= threshold else "REAL",
    "confidence_fake": fusion_score * 100,
    "confidence_real": (1 - fusion_score) * 100,
    "modalities": list(scores.keys())
}

```

6.6.2 Result display function

```

def display_fusion_result(result):

    print("Verdict :", result["verdict"])
    print("Fake Confidence :", result["confidence_fake"])
    print("Real Confidence :", result["confidence_real"])

    plt.bar(
        ["Fake", "Real"],
        [
            result["confidence_fake"],
            result["confidence_real"]
        ]
    )
    plt.ylabel("Confidence (%)")
    plt.show()

```

6.7 Notebook 6: Streamlit Web Application

6.7.1 Imports and Page Configuration

```
import streamlit as st

import torch, torch.nn as nn, numpy as np, cv2, os, tempfile

from PIL import Image

from torchvision import transforms, models

from transformers import BertTokenizer, BertForSequenceClassification

from pytorch_grad_cam import GradCAM

st.set_page_config(
    page_title='FakeDetect',
    page_icon='👁',
    layout='wide',
    initial_sidebar_state='collapsed'
)
```

6.7.2 Loading the Trained Models

```
@st.cache_resource
def load_models():
    tok = BertTokenizer.from_pretrained(BERT_PATH)
    bert = BertForSequenceClassification.from_pretrained(BERT_PATH).to(DEVICE)
    bert.eval()
    m = models.efficientnet_b0(weights=None)
    inf = m.classifier[1].in_features
    m.classifier = nn.Sequential(
        nn.Dropout(0.3), nn.Linear(inf,512),
        nn.ReLU(), nn.Dropout(0.15), nn.Linear(512,2))
    w = torch.load(CNN_PATH, map_location=DEVICE)
    m.load_state_dict(w['model_state'] if 'model_state' in w else w)
    m.to(DEVICE).eval()
    return tok, bert, m
```

6.7.3 Text Prediction Function

```
def pred_text(text, tok, bert):  
    e = tok(text.lower().strip(), max_length=128, truncation=True,  
padding='max_length', return_tensors='pt')  
  
    with torch.no_grad():  
        p = torch.softmax(bert(input_ids=e['input_ids'].to(DEVICE),  
attention_mask=e['attention_mask'].to(DEVICE)).logits, dim=1)  
        return p[0][1].item()
```

6.7.4 Image Prediction with TTA

```
def pred_img_tta(pil_img, cnn):  
    img = pil_img.convert('RGB')  
    probs = []  
    for tfm in [base_tfm, flip_tfm, crop_tfm]:  
        t = tfm(img).unsqueeze(0).to(DEVICE)  
        with torch.no_grad():  
            p = torch.softmax(cnn(t), dim=1)  
            probs.append(p[0].cpu().numpy())  
    avg = np.mean(probs, axis=0)  
    return avg[0], avg[1]
```

6.7.5 Grad-CAM Region Overlay

```
def get_gradcam_overlay(pil_img, cnn):  
    input_tensor = base_tfm(pil_img).unsqueeze(0).to(DEVICE)  
  
    cam = GradCAM(model=cnn, target_layers=[cnn.features[-1]] )  
    grayscale_cam = cam(input_tensor=input_tensor)[0]  
  
    return grayscale_cam
```

6.7.6 Video Prediction with OpenCV

```
def pred_vid(path, cnn, n=20):
    cap = cv2.VideoCapture(path)
    probs = []
    for i in np.linspace(0, int(cap.get(cv2.CAP_PROP_FRAME_COUNT))-1, n, dtype=int):
        cap.set(cv2.CAP_PROP_POS_FRAMES, i)
        ret, frame = cap.read()
        if ret:
            img = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) )
            fake, _ = pred_img_tta(img, cnn)
            probs.append(fake)
    cap.release()
    return np.mean(probs)
```

6.7.7 Multimodal Fusion Logic

```
def show_result(scores, weights):
    tw = sum(weights[k] for k in scores)
    nw = {k: weights[k]/tw for k in scores}
    fs = sum(scores[k]*nw[k] for k in scores)
    fp = fs * 100
    rp = (1-fs) * 100
    verdict = "FAKE DETECTED" if fs >= 0.5 else "REAL CONTENT"
    return verdict, fp, rp
```

CHAPTER – 7

OUTPUT SNAPSHOTS

7.1 Fake News Detection Outputs

7.1.1 Preview of the Fake and Real News Dataset

The Fake and Real News datasets were successfully loaded into the system. The following snapshot presents sample records from the fake and real news datasets used for training the BERT model.

Figure 7.1: Sample records from the Fake and Real News datasets.

```
=====
FAKE NEWS DATASET
=====
Shape      : (23481, 4)
Columns    : ['title', 'text', 'subject', 'date']
Null vals  : {'title': 0, 'text': 0, 'subject': 0, 'date': 0}

Sample rows:
```

	title	text	subject	date
0	Donald Trump Sends Out Embarrassing New Year'...	Donald Trump just couldn t wish all Americans ...	News	December 31, 2017
1	Drunk Bragging Trump Staffer Started Russian ...	House Intelligence Committee Chairman Devin Nu...	News	December 31, 2017
2	Sheriff David Clarke Becomes An Internet Joke...	On Friday, it was revealed that former Milwauk...	News	December 30, 2017

```
=====
TRUE NEWS DATASET
=====
Shape      : (21417, 4)
Columns    : ['title', 'text', 'subject', 'date']
Null vals  : {'title': 0, 'text': 0, 'subject': 0, 'date': 0}

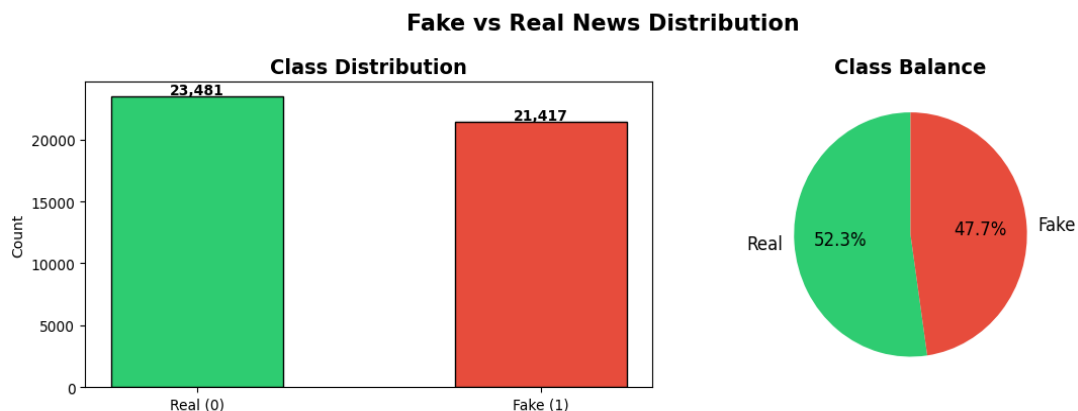
Sample rows:
```

	title	text	subject	date
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	December 31, 2017
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	December 29, 2017
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	December 31, 2017

7.1.2 Distribution of Fake and Real News Articles

The class distribution illustrates the number and proportion of fake and real news articles available in the dataset.

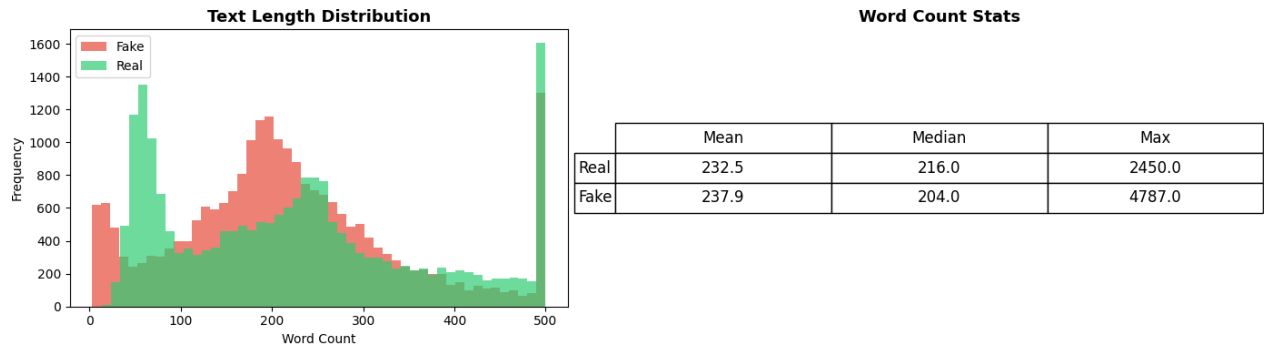
Figure 7.2: Class-wise distribution of fake and real news articles.



7.1.3 Distribution of News Article Lengths

The graph shows the variation in the lengths of news articles included in the dataset after preprocessing.

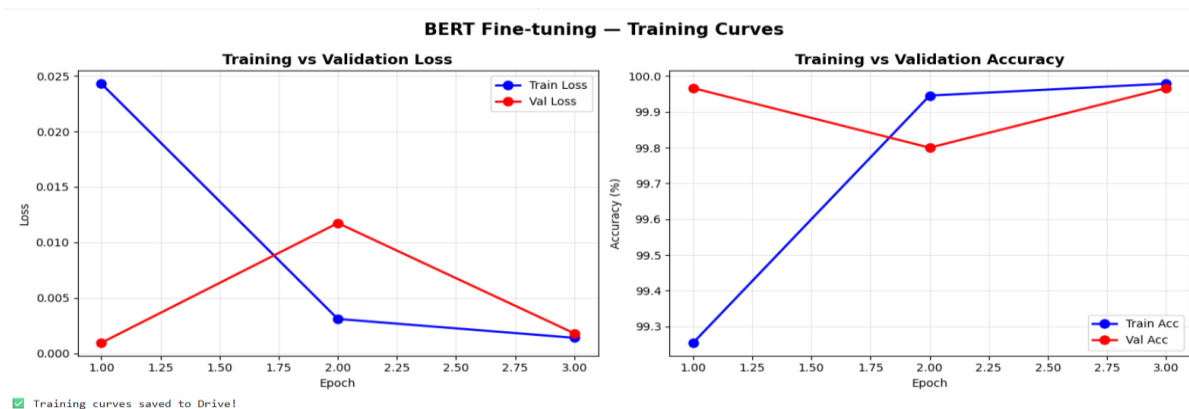
Figure 7.3: Distribution of article lengths in the news dataset.



7.1.4 BERT Training and Validation Performance

The training curves demonstrate the learning progress of the BERT model across multiple training epochs.

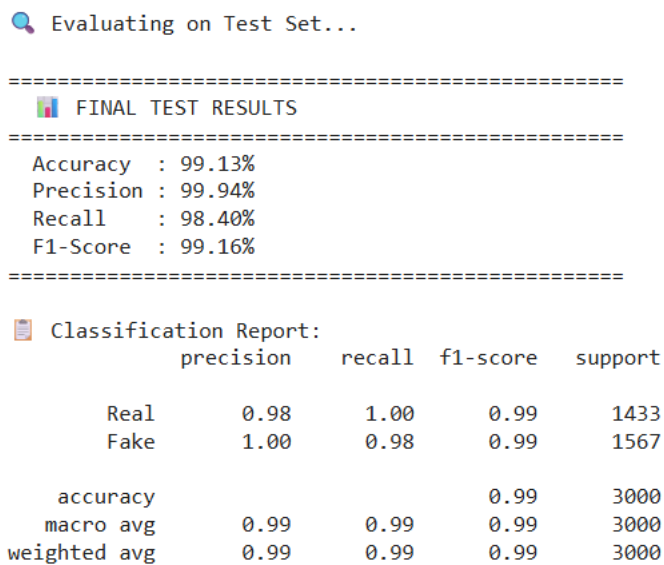
Figure 7.4: Training and validation accuracy/loss of the BERT model.



7.1.5 BERT Classification Report

The classification report summarizes the performance of the BERT model using precision, recall, F1-score, and accuracy.

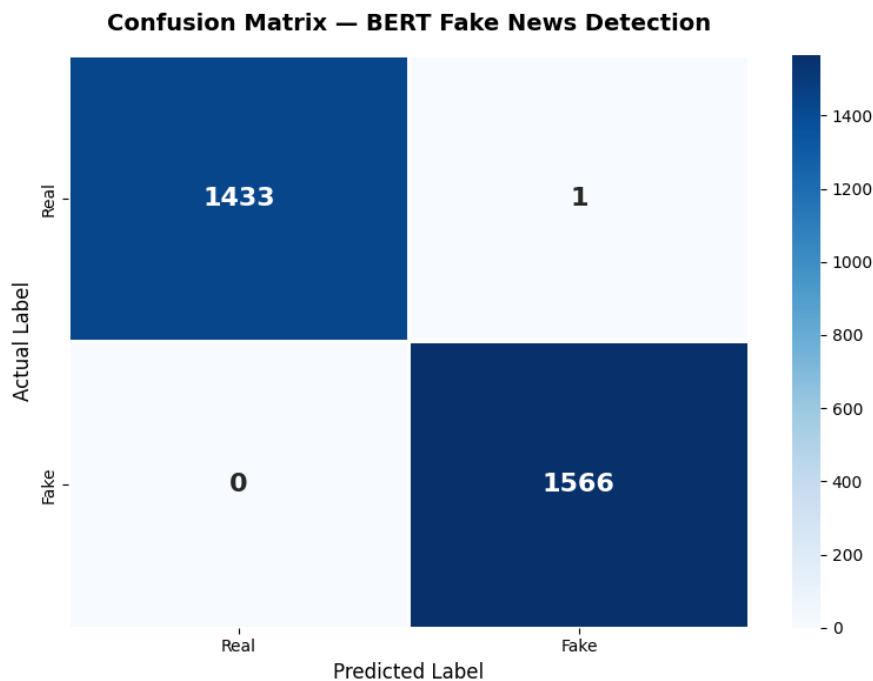
Figure 7.5: Performance metrics of the BERT fake news detection model.



7.1.6 Confusion Matrix of the BERT Model

The confusion matrix illustrates the classification results by comparing the predicted and actual labels.

Figure 7.6: Confusion matrix of the BERT fake news detection model.



7.1.7 Sample Predictions Generated by the BERT Model

The following examples demonstrate the predictions produced by the trained BERT model on sample news articles.

Figure 7.7: Sample prediction results generated by the BERT model.

```
=====
🔍 FAKE NEWS DETECTION RESULTS
=====

[1] The United States Federal Reserve raised interest rates by 25 basis po...
Prediction : ● REAL
Real      : 97.1% | Fake: 2.9%
Confidence : HIGH

-----

[2] NASA scientists have confirmed that the moon is slowly moving away fro...
Prediction : ● REAL
Real      : 60.4% | Fake: 39.6%
Confidence : MODERATE

-----

[3] SHOCKING: Bill Gates admits in secret video that COVID-19 vaccines con...
Prediction : ● FAKE
Real      : 0.0% | Fake: 100.0%
Confidence : HIGH

-----

[4] The Indian Space Research Organisation successfully launched its PSLV-...
Prediction : ● REAL
Real      : 96.8% | Fake: 3.2%
Confidence : HIGH

-----

[5] Scientists at MIT have PROVEN that drinking a mixture of turmeric, lem...
Prediction : ● FAKE
Real      : 0.1% | Fake: 99.9%
Confidence : HIGH

-----

[6] The European Central Bank announced a package of economic measures aim...
Prediction : ● REAL
Real      : 98.2% | Fake: 1.8%
Confidence : HIGH

-----

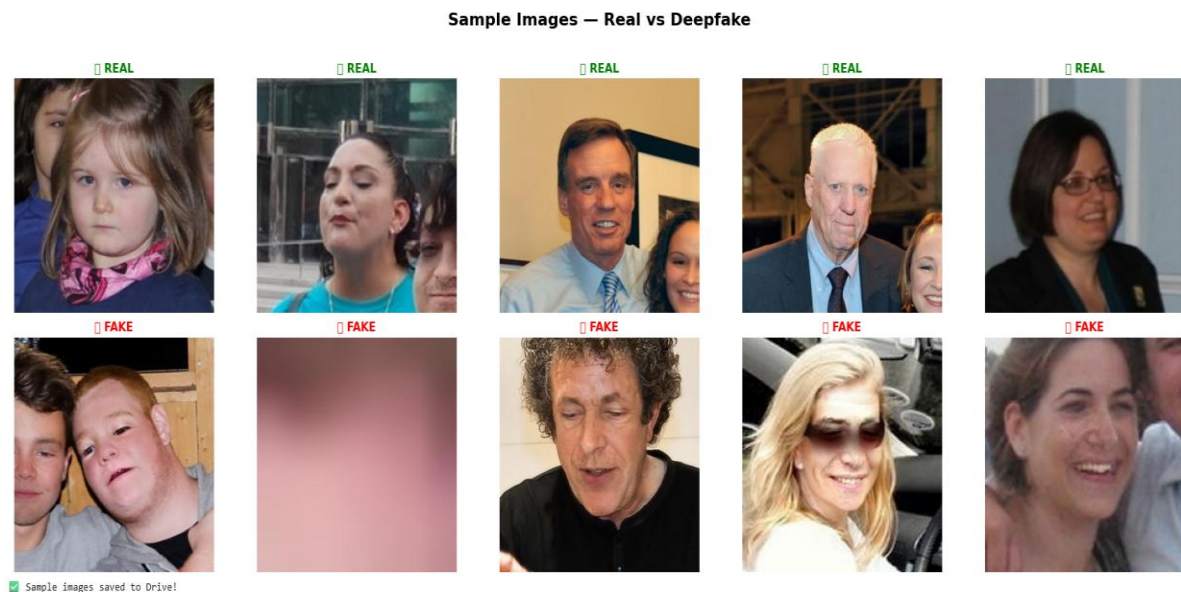
[7] URGENT: The government has secretly passed a law that allows them to s...
Prediction : ● FAKE
Real      : 3.7% | Fake: 96.3%
Confidence : HIGH
```

7.2 Deepfake Image Detection Outputs

7.2.1 Sample Images from the Deepfake Dataset

The following images represent sample real and deepfake images used for training and evaluation.

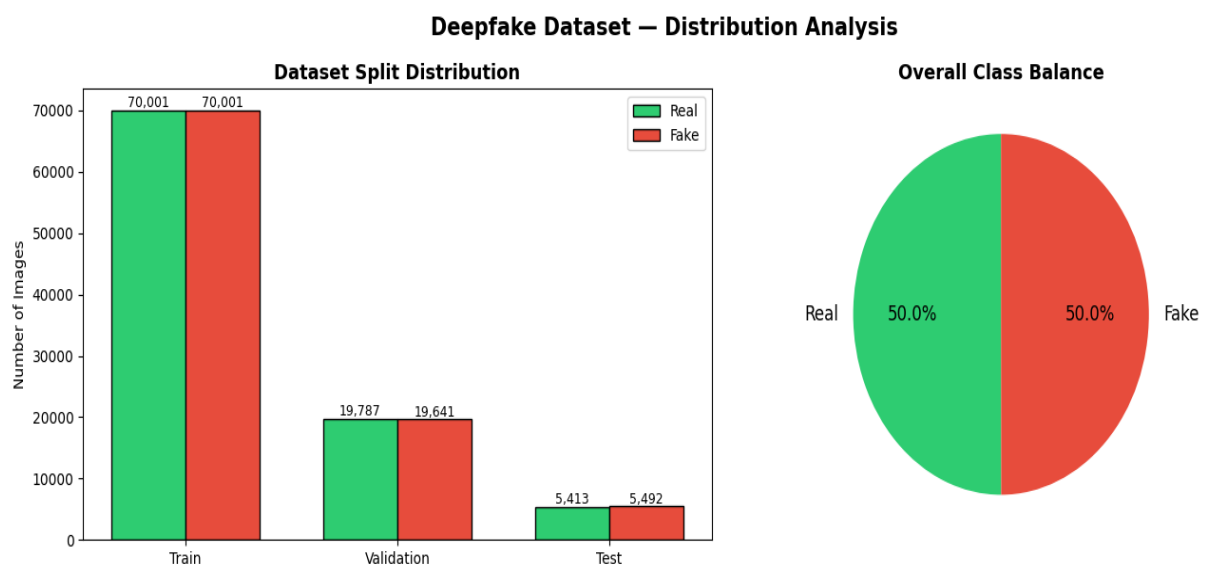
Figure 7.8: Sample images from the deepfake image dataset.



7.2.2 Distribution of Real and Fake Images in the Deepfake Dataset

The chart presents the distribution of real and fake images available in the deepfake dataset.

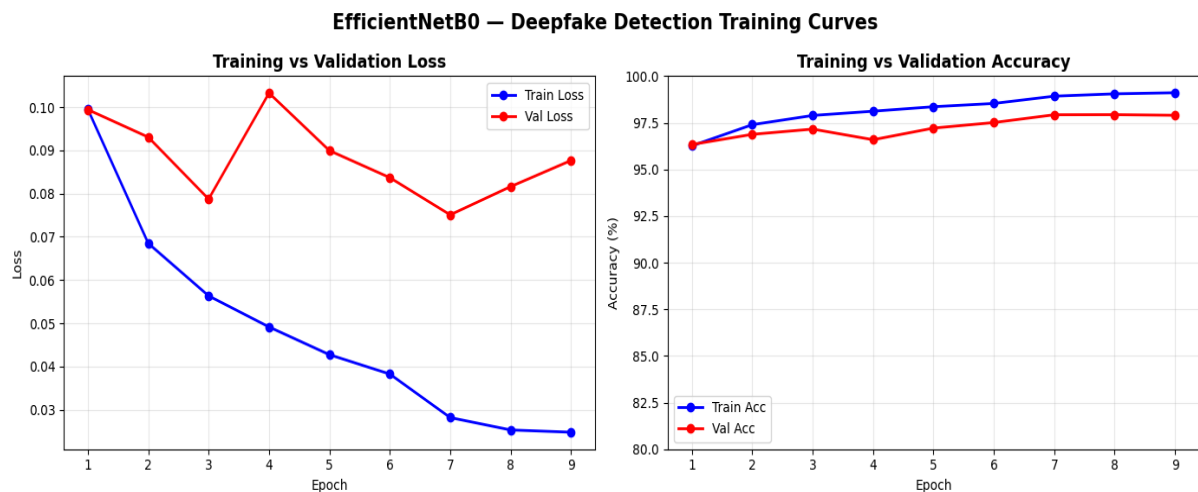
Figure 7.9: Class distribution of real and fake images.



7.2.3 Training and Validation Performance of EfficientNetB0

The learning curves show the training and validation performance of the EfficientNetB0 model during training.

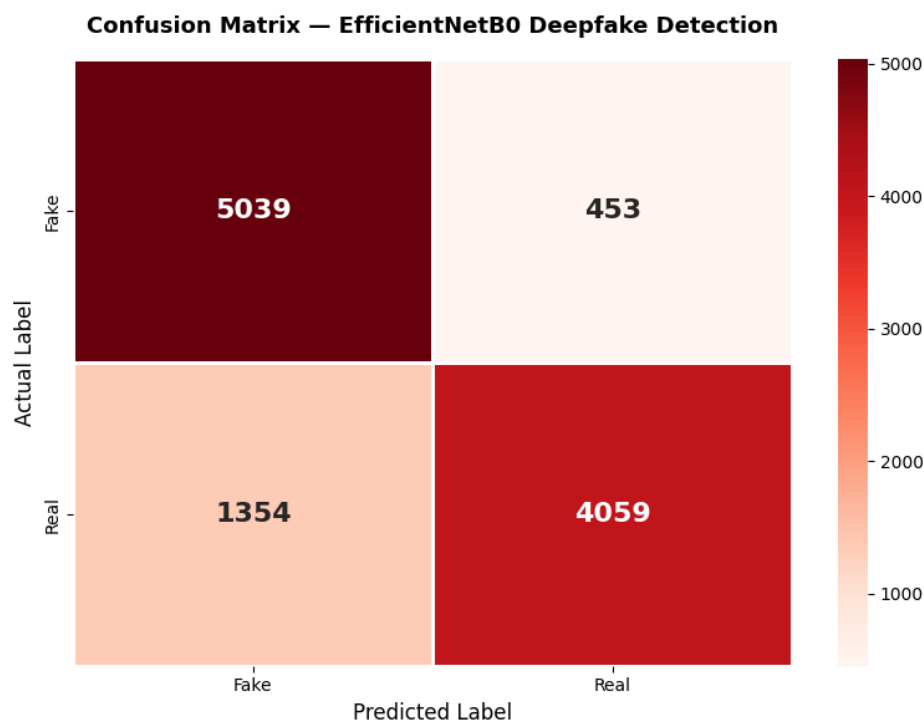
Figure 7.10: Training and validation accuracy/loss of the EfficientNetB0 model.



7.2.4 Confusion Matrix of the EfficientNetB0 Model

The confusion matrix provides an overview of the prediction performance of the EfficientNetB0 model.

Figure 7.11: Confusion matrix of the EfficientNetB0 image detection model.



7.2.5 Classification Report of the Image Detection Model

The classification report summarizes the evaluation metrics obtained for the image deepfake detection model.

Figure 7.12: Performance metrics of the image deepfake detection model.

 Evaluating with Test Time Augmentation (TTA)...
(Takes ~15-20 min for 10K test images)

=====

 TTA TEST RESULTS – CNN DEEPFAKE

=====

Accuracy : 84.54%

Precision : 96.46%

Recall : 71.48%

F1-Score : 82.11%

=====

	precision	recall	f1-score	support
Fake	0.78	0.97	0.86	5492
Real	0.96	0.71	0.82	5413
accuracy			0.85	10905
macro avg	0.87	0.84	0.84	10905
weighted avg	0.87	0.85	0.84	10905

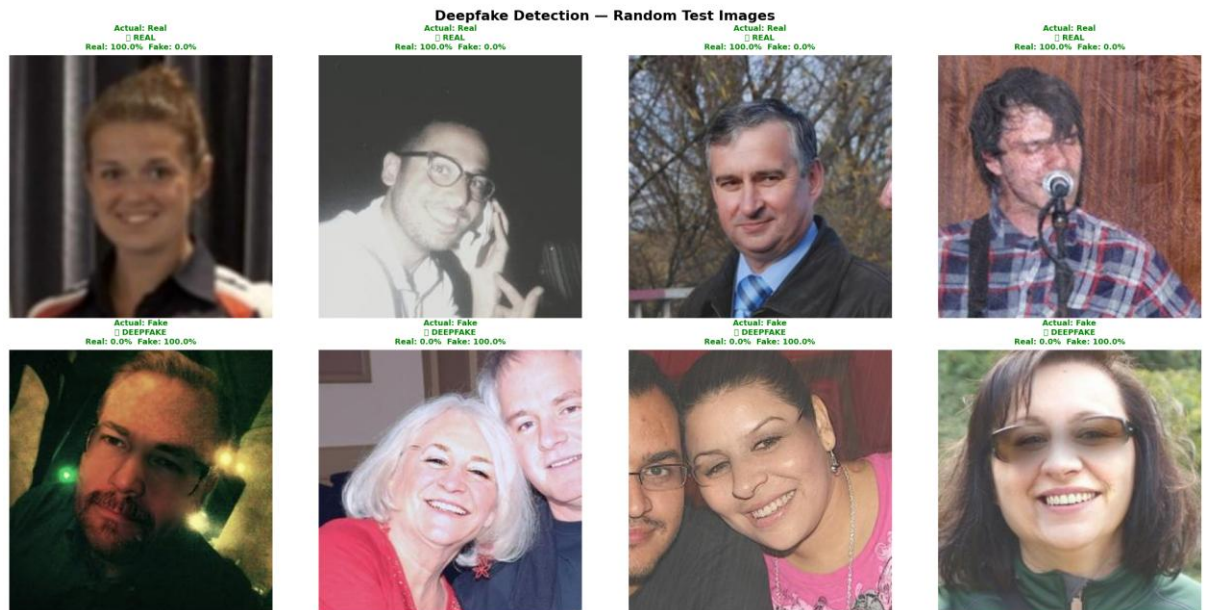
=====

7.2.6 Image Prediction Using EfficientNetB0

The following output demonstrates the prediction generated by the EfficientNetB0 model for a sample test image along with its confidence score.

Figure 7.13: Prediction result of the EfficientNetB0 model for a test image.

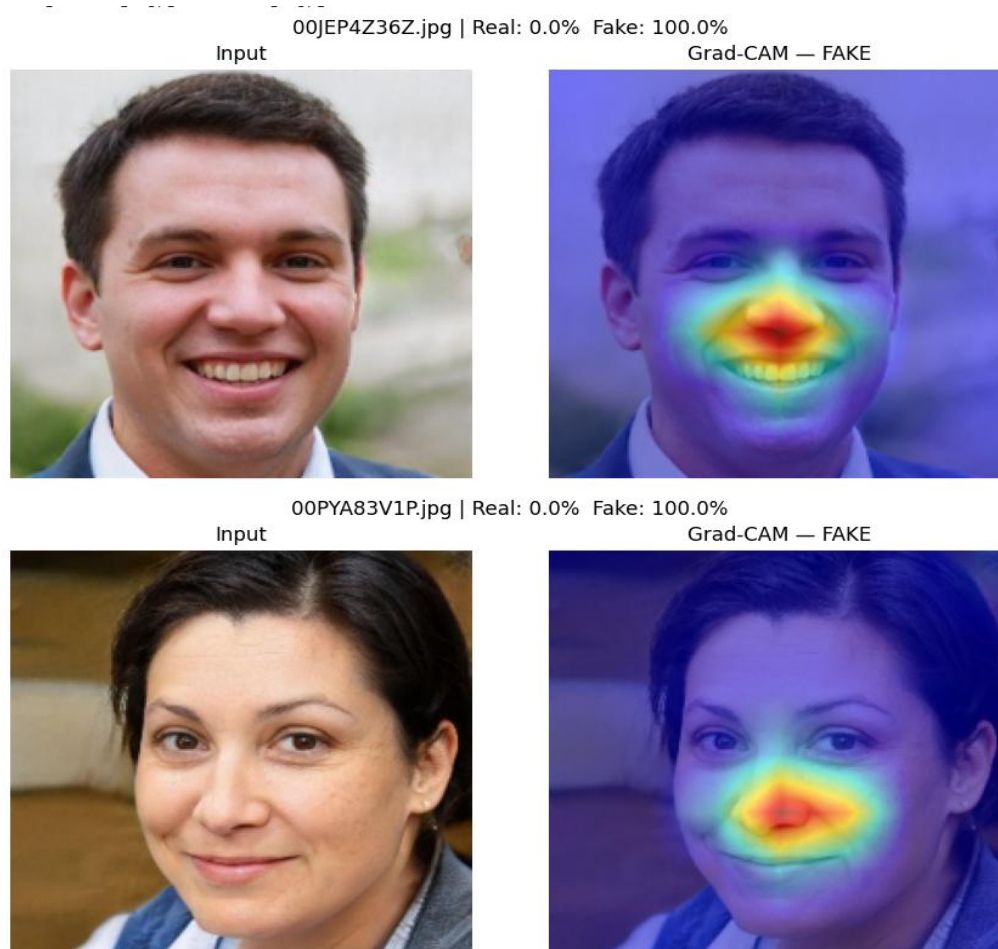




7.2.7 GRAD-CAM

The Grad-CAM visualization highlights the image regions that contributed most to the deepfake prediction, improving model interpretability.

Figure 7.14: Grad-CAM heatmap showing the important regions used for prediction.

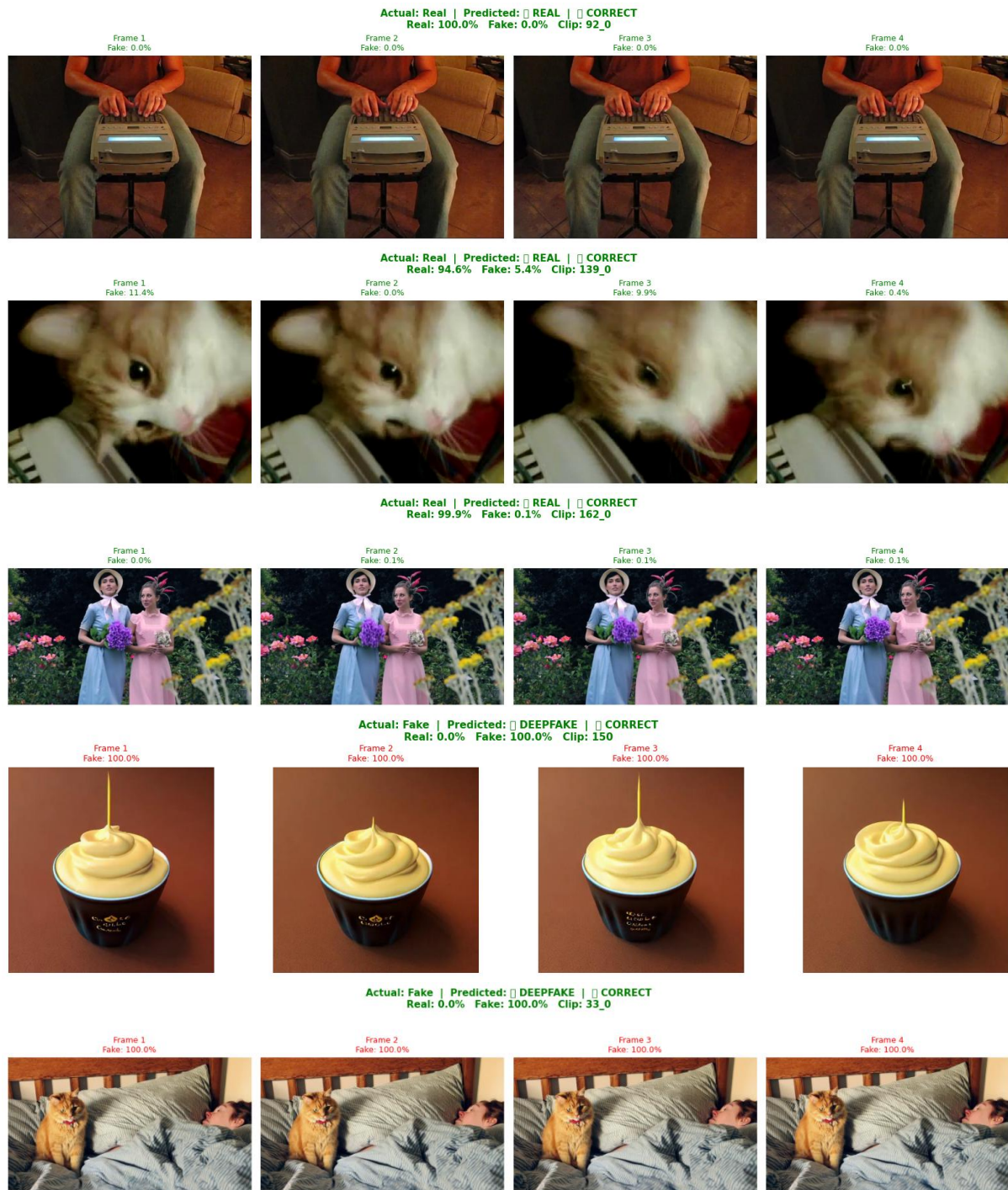


7.3 Deepfake Video Detection Outputs

7.3.1 Sample Video Frames from the Deepfake Dataset

Representative frames extracted from the input video are used for deepfake analysis.

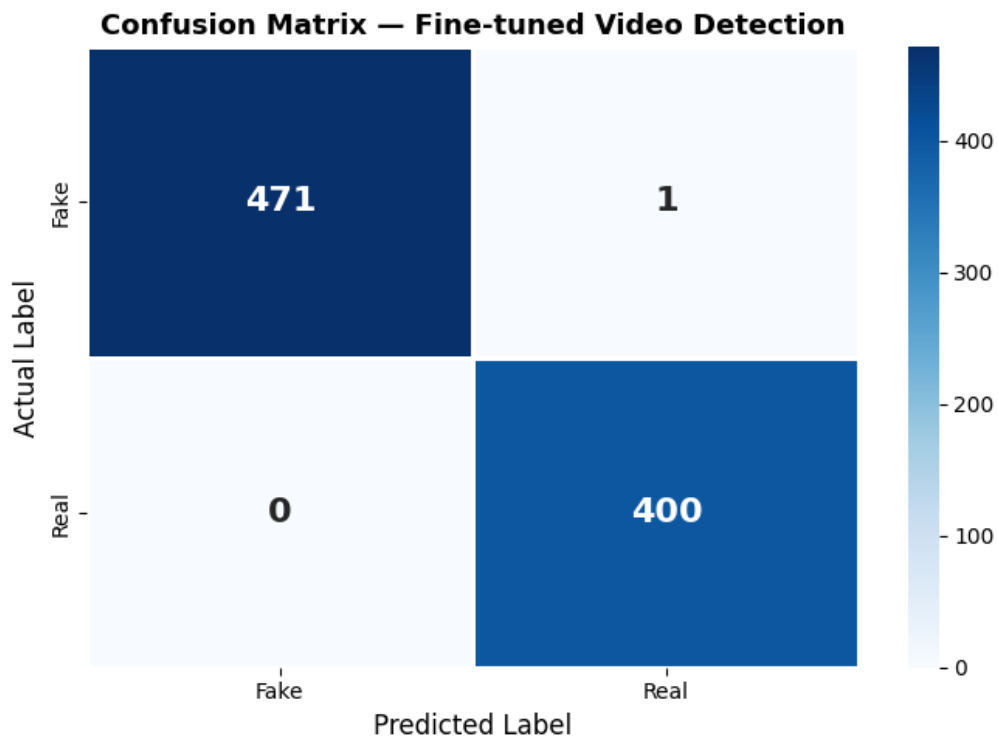
Figure 7.15: Sample frames extracted from the input video.



7.3.2 Confusion Matrix of the Video Detection Model

The confusion matrix summarizes the classification performance of the video deepfake detection model.

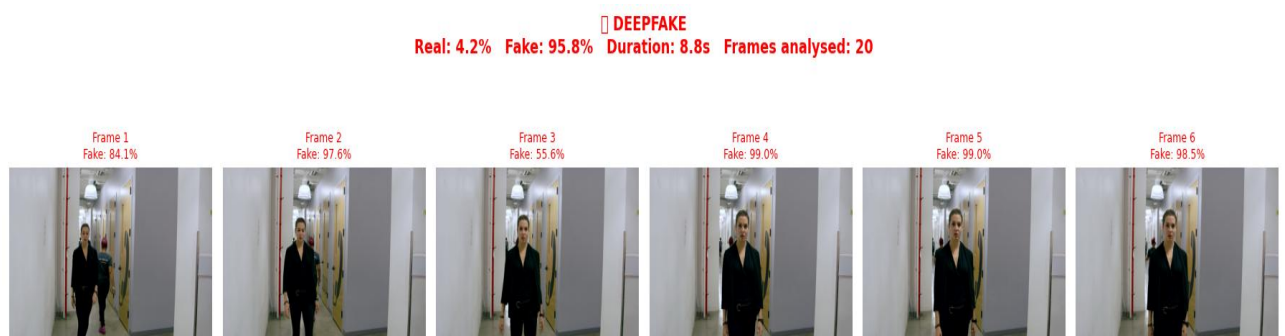
Figure 7.16: Confusion matrix of the video deepfake detection model.



7.3.3 Video Deepfake Detection Result

The system predicts whether the uploaded video is real or manipulated based on frame-level analysis.

Figure 7.17: Prediction result for a sample input video.

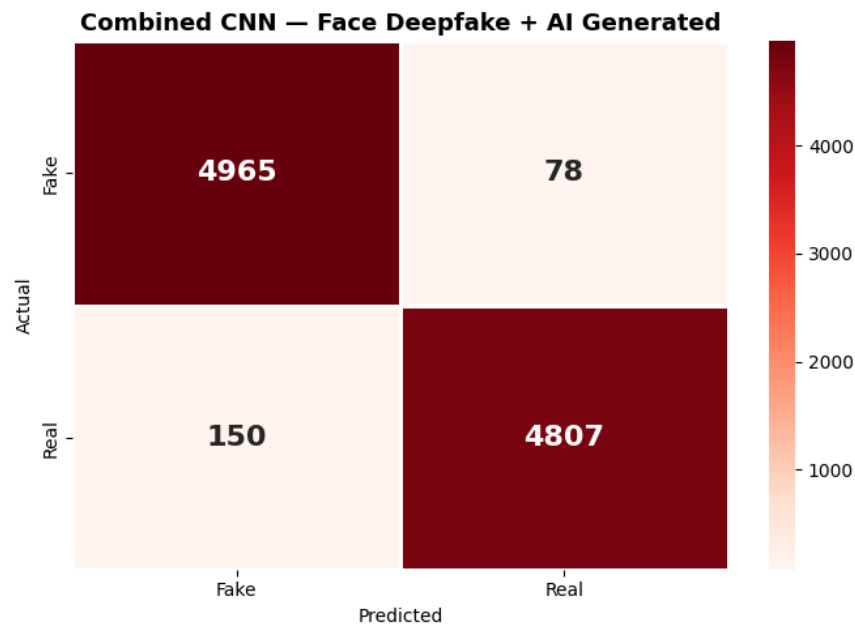


7.4 Combined CNN Model (CIFAKE, CGI, AI Generated, FaceSwap Images) Outputs

7.4.1 Combined CNN (Face Deepfake + AI-Generated) – Confusion Matrix

The confusion matrix presents the performance of the combined CNN model trained on multiple image datasets.

Figure 7.18: Confusion matrix of the combined CNN model.



7.4.2 Combined CNN – Classification Report

The classification report presents the precision, recall, F1-score, and accuracy achieved by the combined CNN model.

Figure 7.19: Performance metrics of the combined CNN model.

```
Evaluating on CIFAKE test set...

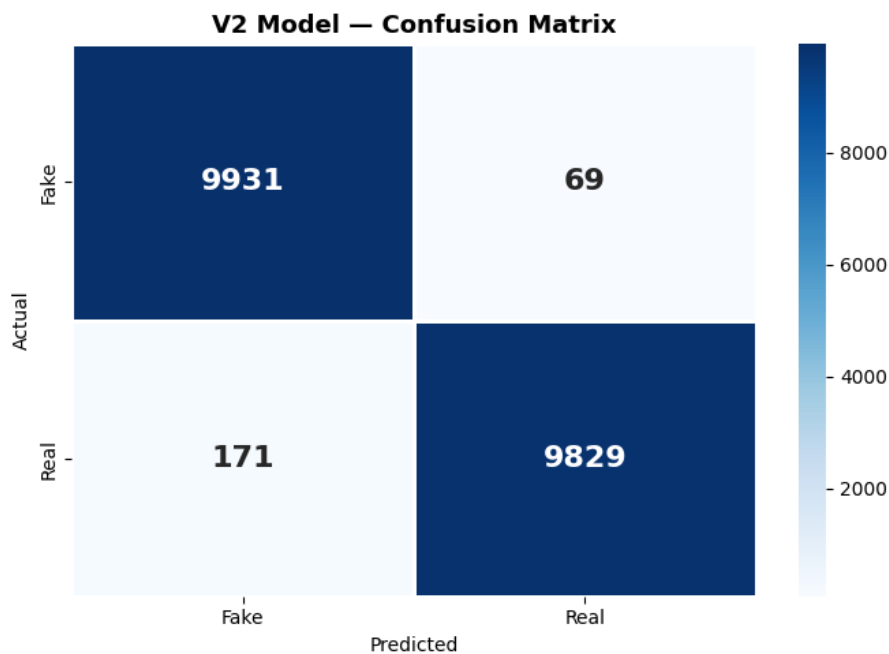
=====
COMBINED CNN – FINAL TEST RESULTS
=====
Accuracy : 97.72%
Precision : 98.40%
Recall : 96.97%
F1-Score : 97.68%
=====
```

	precision	recall	f1-score	support
Fake	0.97	0.98	0.98	5043
Real	0.98	0.97	0.98	4957
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

7.4.3 EfficientNetB0 V2 – Confusion Matrix

The confusion matrix illustrates the prediction performance of the EfficientNetB0 V2 model.

Figure 7.20: Confusion matrix of the EfficientNetB0 V2 model.



7.4.4 EfficientNetB0 V2 – Classification Report

The classification report summarizes the evaluation metrics obtained by the EfficientNetB0 V2 model.

Figure 7.21: Performance metrics of the EfficientNetB0 V2 model.

```
Final evaluation on test set...
```

V2 MODEL – FINAL TEST RESULTS

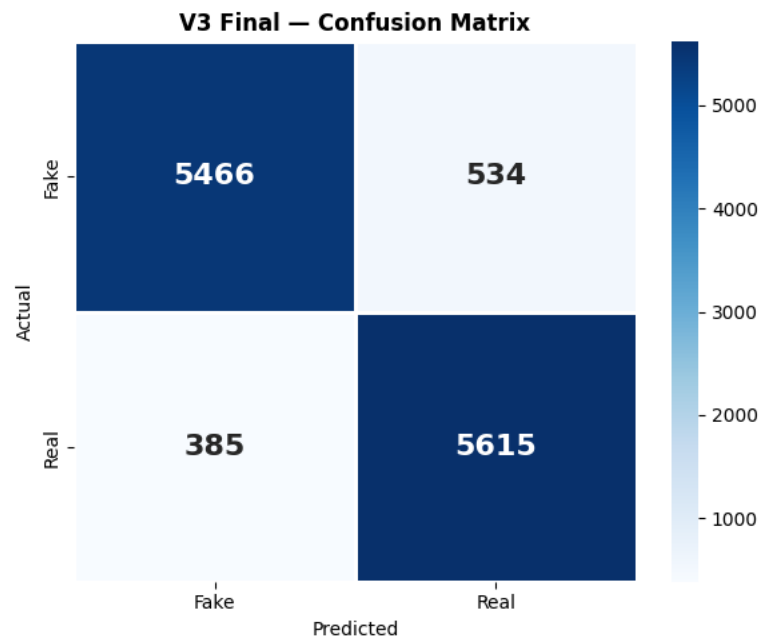
```
Accuracy : 98.80%
Precision : 99.30%
Recall    : 98.29%
F1-Score  : 98.79%
```

	precision	recall	f1-score	support
Fake	0.98	0.99	0.99	10000
Real	0.99	0.98	0.99	10000
accuracy			0.99	20000
macro avg	0.99	0.99	0.99	20000
weighted avg	0.99	0.99	0.99	20000

7.4.5 EfficientNetB0 V3 – Confusion Matrix

The confusion matrix illustrates the prediction accuracy of the final EfficientNetB0 V3 model.

Figure 7.22: Confusion matrix of the EfficientNetB0 V3 model.



7.4.6 EfficientNetB0 V3 – Classification Report

The classification report presents the final evaluation metrics achieved by the EfficientNetB0 V3 model.

Figure 7.23: Performance metrics of the EfficientNetB0 V3 model.

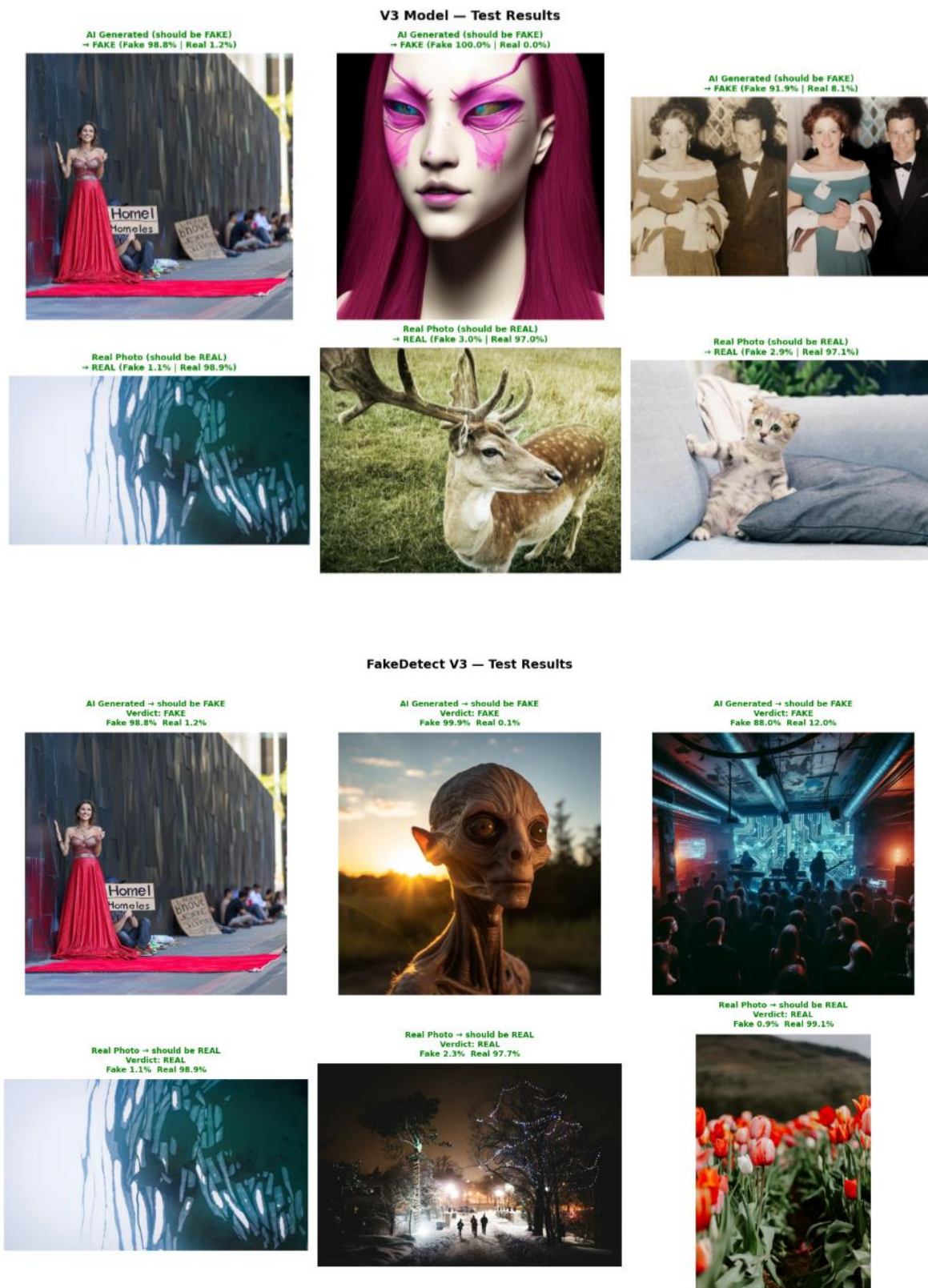
```
=====
V3 MODEL – FINAL TEST RESULTS
=====
Accuracy : 92.34%
Precision: 91.32%
Recall   : 93.58%
F1-Score : 92.44%
=====
```

	precision	recall	f1-score	support
Fake	0.93	0.91	0.92	6000
Real	0.91	0.94	0.92	6000
accuracy			0.92	12000
macro avg	0.92	0.92	0.92	12000
weighted avg	0.92	0.92	0.92	12000

7.4.7 Sample Predictions (Real and Fake Images)

Sample test images are classified as real or fake along with their prediction confidence.

Figure 7.24: Prediction results for sample real and fake images.

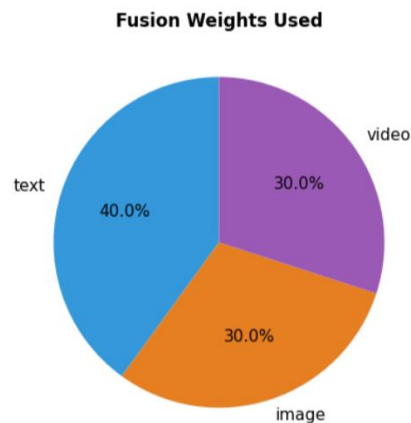


7.5 Multimodal Fusion Outputs

7.5.1 Fusion Weights

The predefined weights assigned to text, image, and video modalities during multimodal fusion are shown below.

Figure 7.25: Weight distribution used in multimodal fusion.

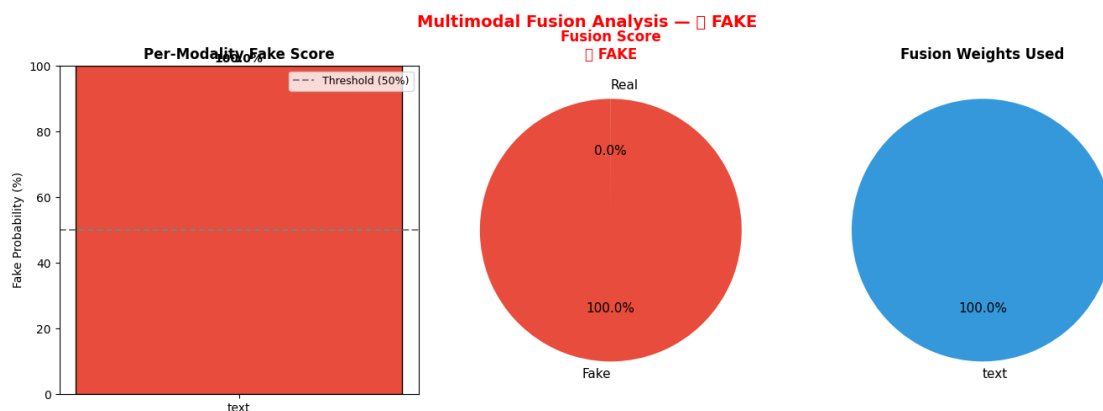


7.5.2 Text-only prediction

The BERT model predicts the authenticity of textual news content without using image or video information.

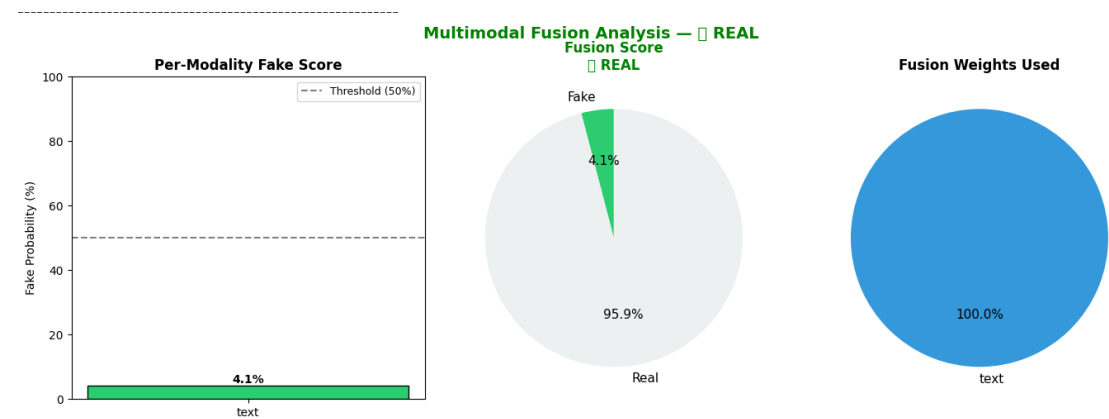
Text: SHOCKING: Scientists have proven that 5G towers are being used by the government to control citizens minds. Secret documents leaked from the WHO confirm that COVID vaccines contain microchips that activate when exposed to 5G signals. Share this before it gets deleted!

Figure 7.26: Prediction result for text-only input.



Text: The Federal Reserve raised interest rates by 25 basis points on Wednesday, as policymakers continued efforts to bring inflation back to the central bank's 2 percent target. The decision was unanimous among committee members.

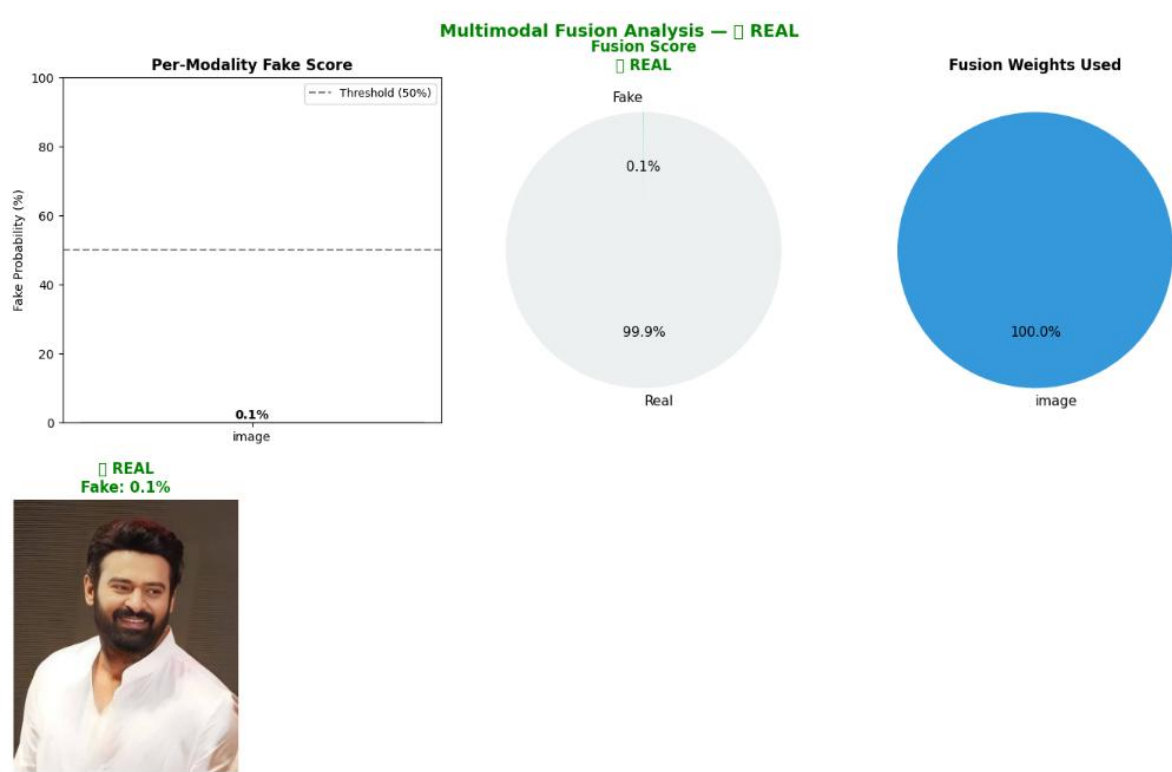
Figure 7.27: Prediction result for text-only input.



7.5.3 Image-only prediction

The CNN model detects whether the uploaded image is genuine or AI-generated.

Figure 7.28: Prediction result for image-only input.

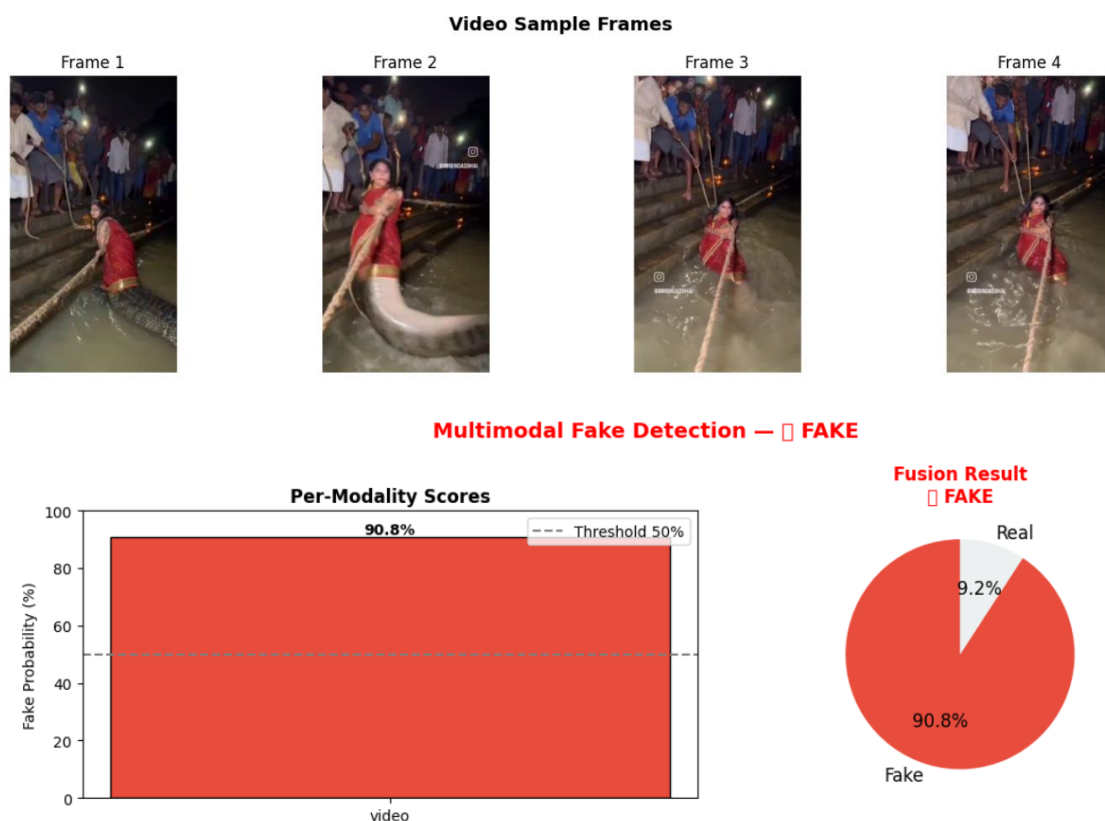




7.5.4 Video-only prediction

The system analyzes video frames to determine whether the uploaded video is authentic or manipulated.

Figure 7.29: Prediction result for video-only input.

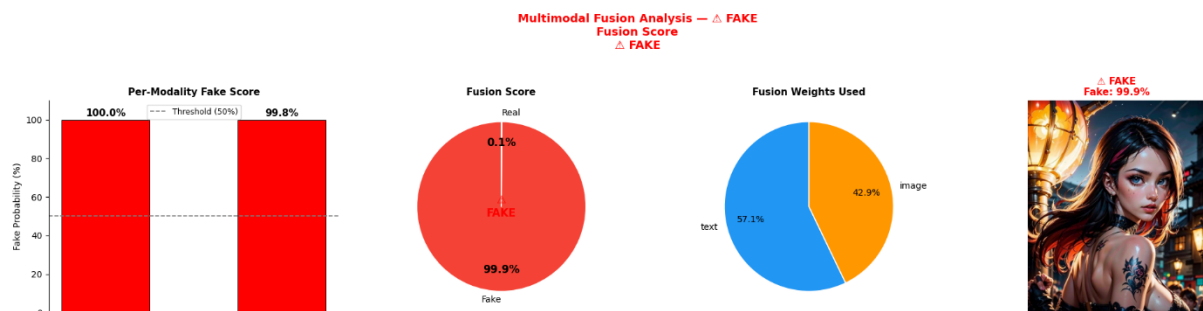


7.5.5 Text + Image prediction

Predictions from both text and image models are combined to generate the final classification.

Text: "Scientists Reveal the World's First AI-Engineered Human Living Among Us"

Figure 7.30: Multimodal prediction using text and image inputs.

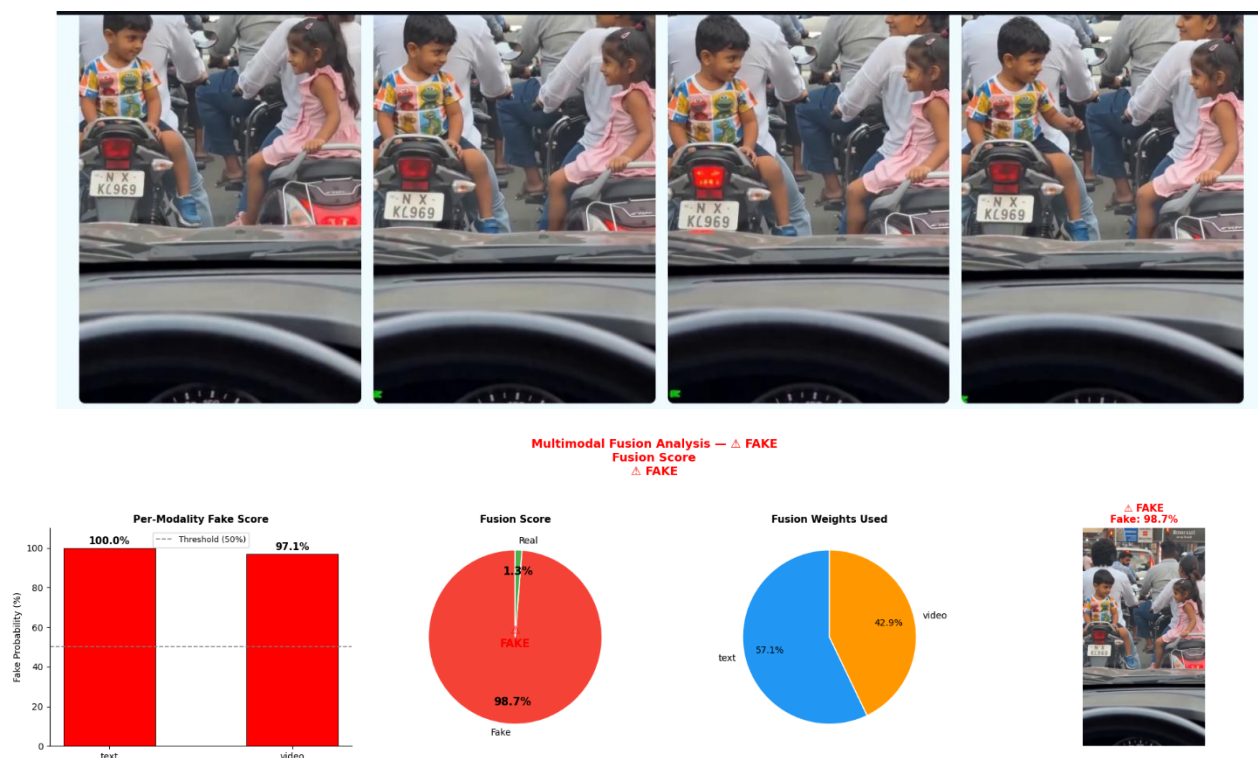


7.5.6 Text + Video prediction

The multimodal system combines textual and video evidence to improve detection accuracy.

Text: Shocking Video Captures Children Floating in the Air During Traffic Jam

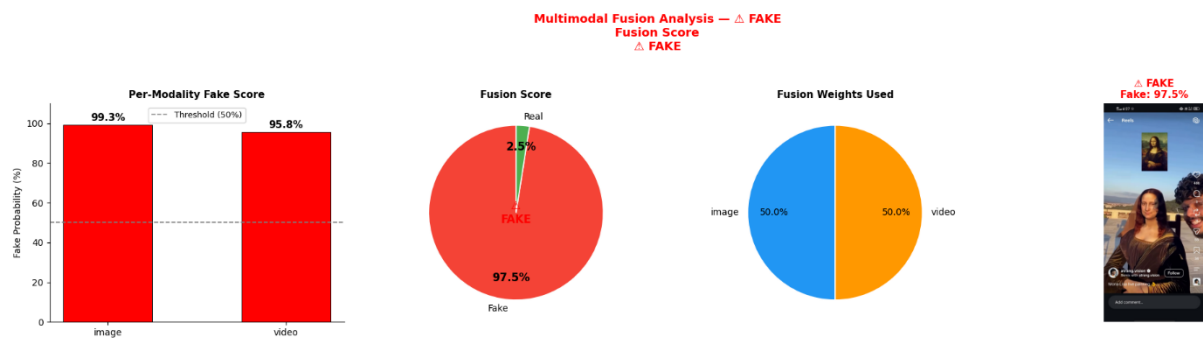
Figure 7.31: Multimodal prediction using text and video inputs.



7.5.7 Image + Video prediction

Image and video predictions are fused to produce a final authenticity score.

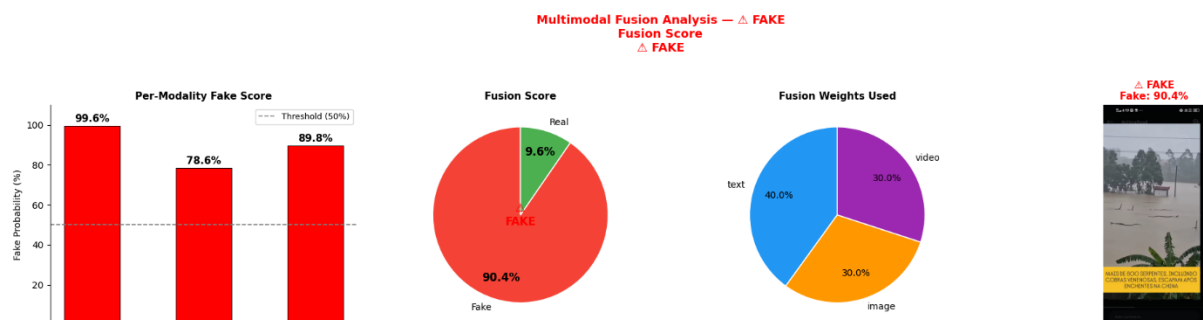
Figure 7.32: Multimodal prediction using image and video inputs.



7.5.8 Text + Image + Video prediction

The final prediction is obtained by combining text, image, and video information through the fusion module.

Figure 7.33: Final multimodal prediction using all three input modalities.

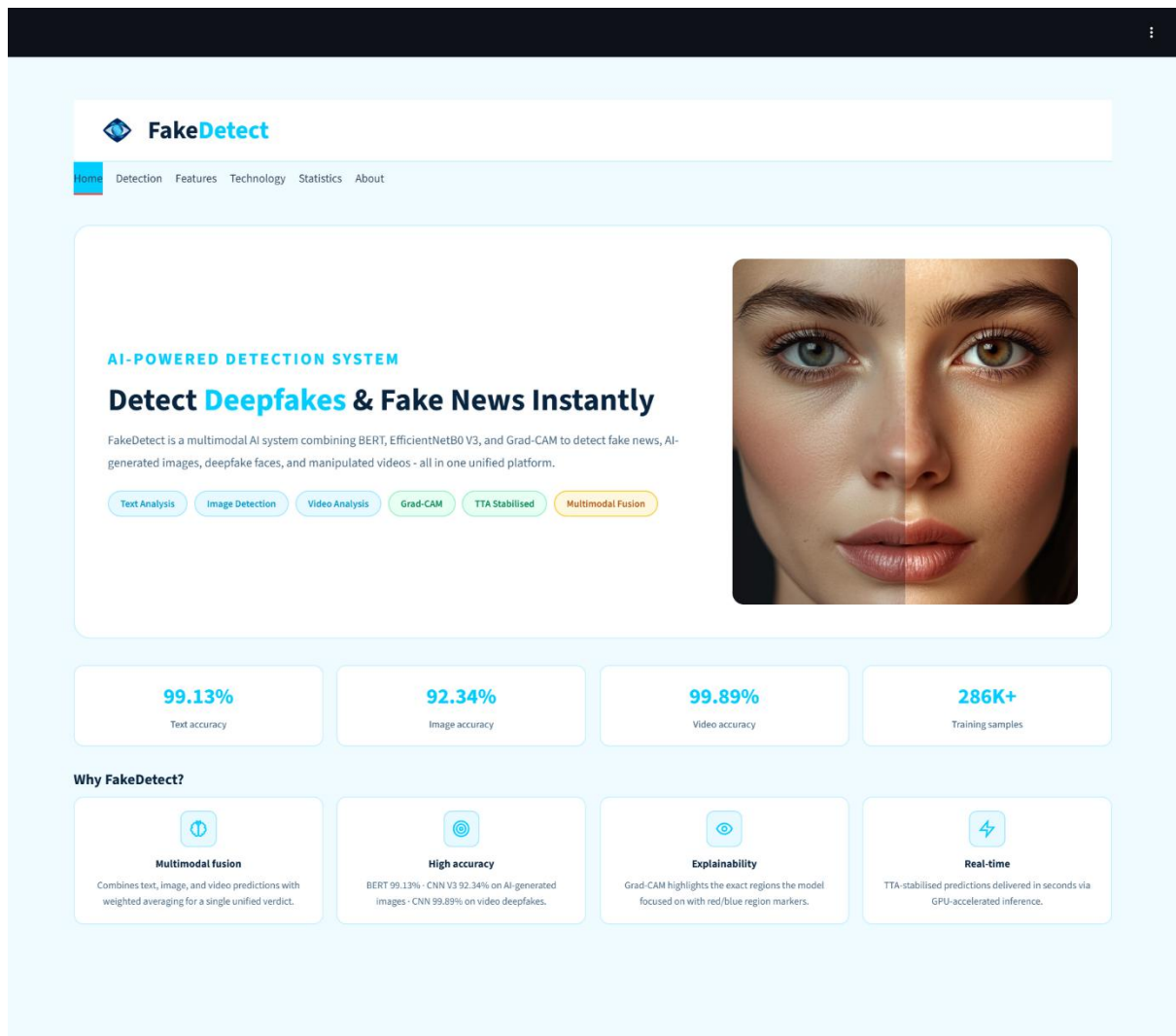


7.6 Streamlit Web Application Snapshots

7.6.1 Home page

The home page provides an overview of the FakeDetect application and its functionality.

Figure 7.34: Home page of the FakeDetect application.



7.6.2 Detection page

Users can upload text, images, or videos for fake content detection through the detection interface.

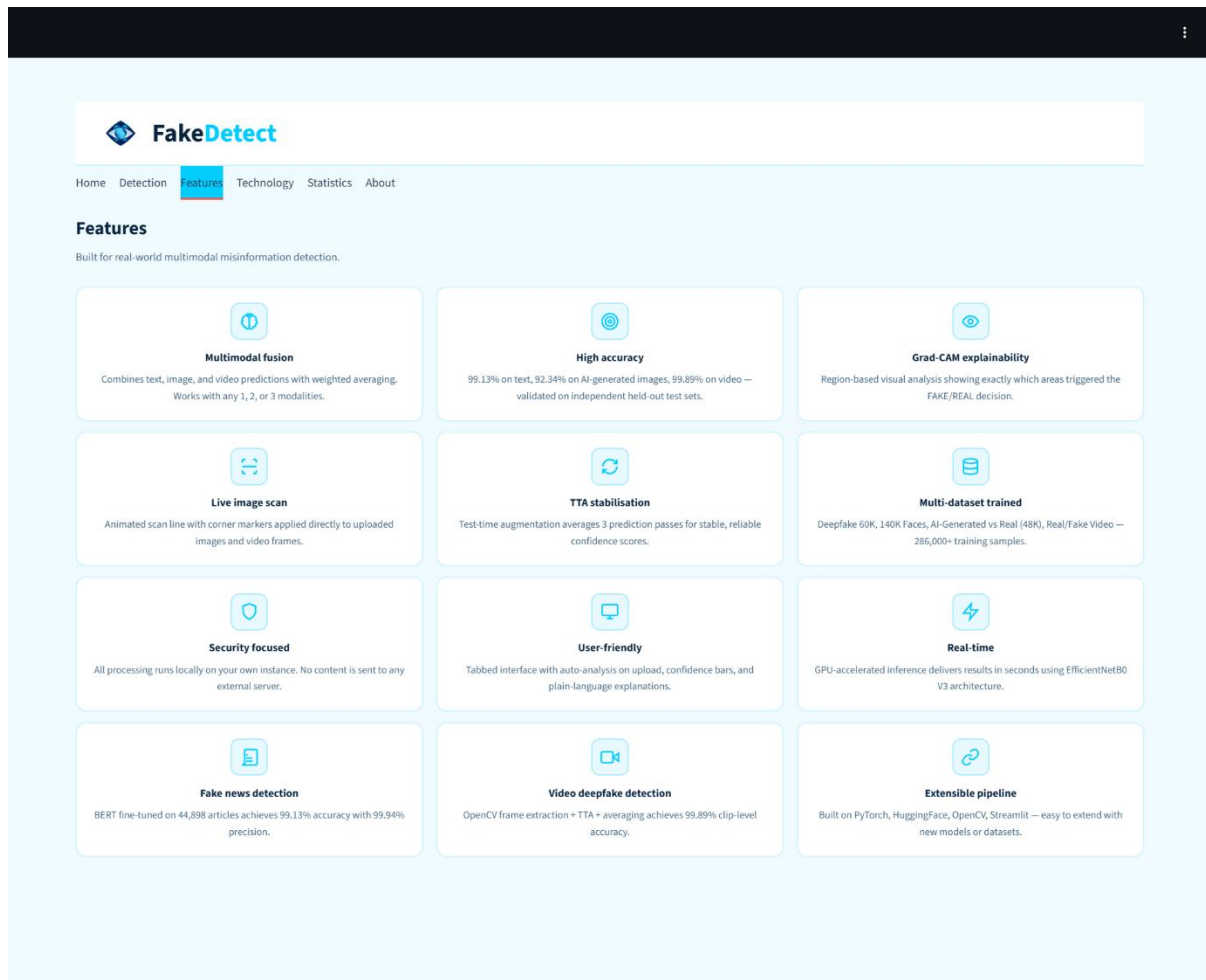
Figure 7.35: Detection interface of the application.

The screenshot shows the 'Detection' page of the FakeDetect application. At the top, there is a navigation bar with links for Home, Detection (highlighted), Features, Technology, Statistics, and About. Below the navigation bar, the 'Detection' section is titled, followed by the instruction 'Upload text, image, or video - results are generated automatically.' A green status bar indicates 'All 3 models loaded and ready!'. Below this, there are tabs for News Text, Image, Video, and Fusion (highlighted). The main heading is 'Multimodal Fusion - Text + Image + Video'. A light blue box contains the text 'Provide any combination. Weights auto-normalise: Text 40% · Image 30% · Video 30%.' Below this, there is a text input field labeled 'News text (optional)'. Underneath, there are two upload sections: 'Image (optional)' and 'Video (optional)'. Each section has an 'Upload' button and specifies a 200MB limit and supported file formats (JPG, PNG, WEBP for images; MP4, AVI, MOV, MKV for videos). At the bottom, there is a blue button labeled 'Run Fusion Detection'.

7.6.3 Features page

The features page summarizes the capabilities provided by the FakeDetect system.

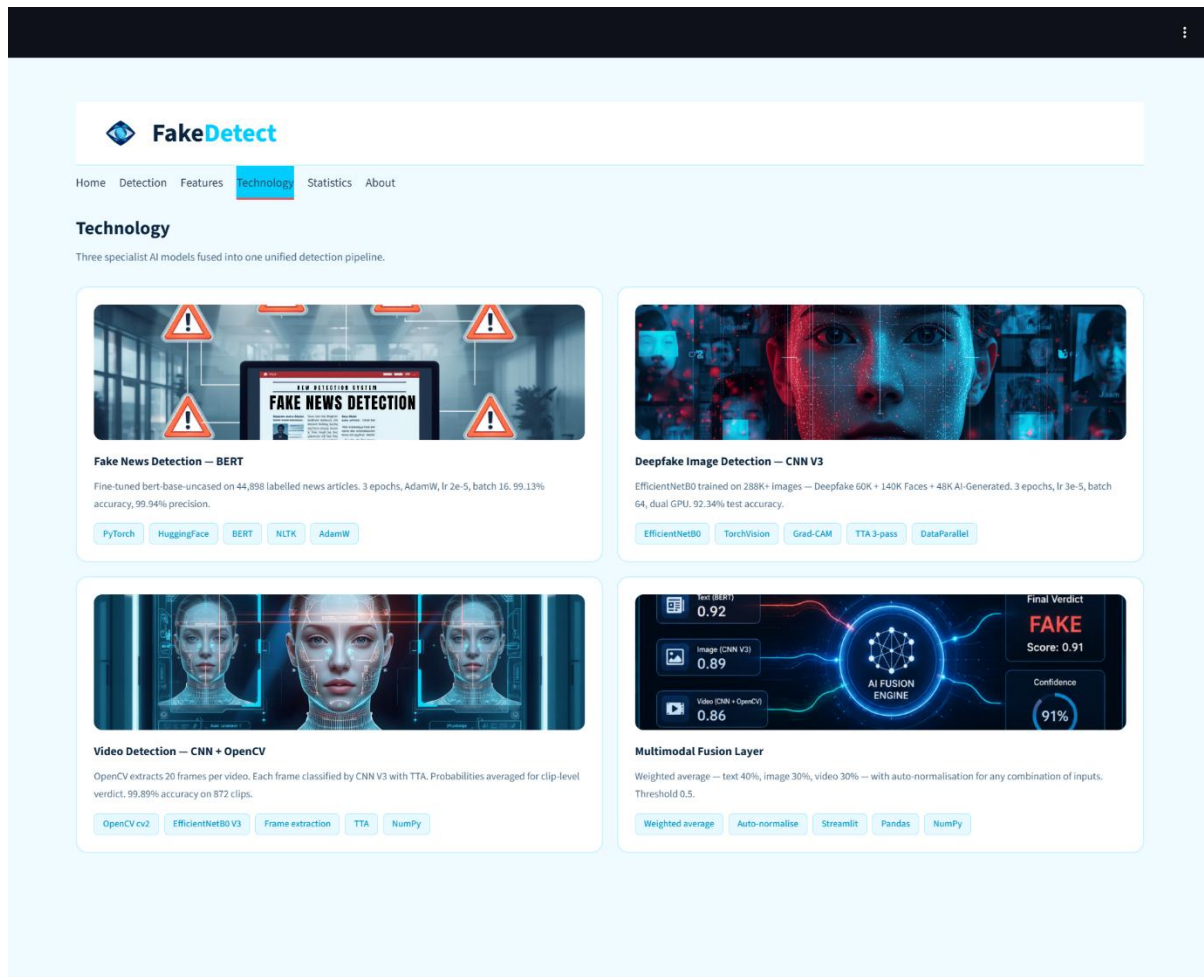
Figure 7.36: Features page of the application.



7.6.4 Technology page

The technologies and deep learning models used in the project are presented on this page.

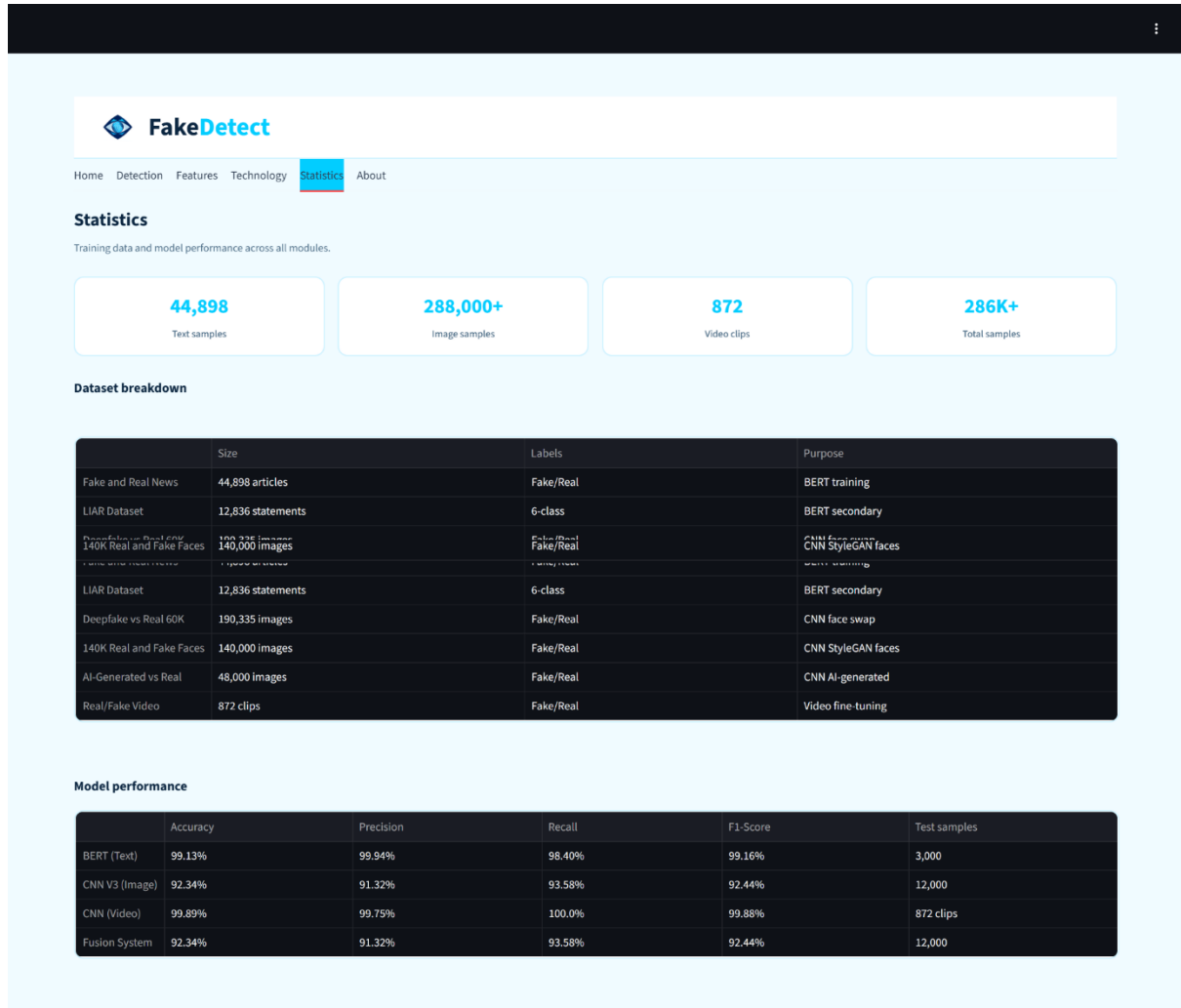
Figure 7.37: Technology page showing the system architecture.



7.6.5 Statistics page

This page displays model performance statistics and evaluation metrics.

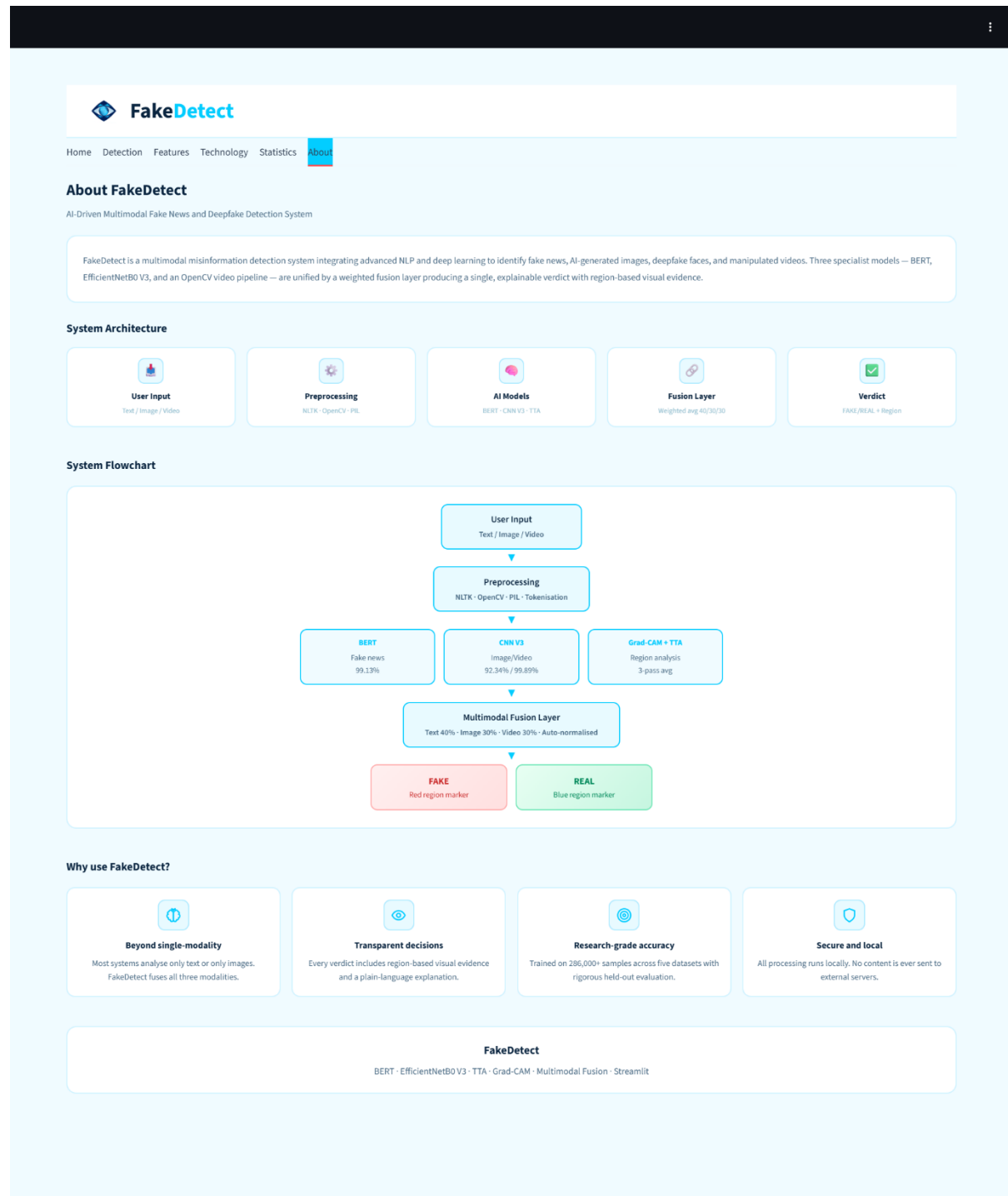
Figure 7.38: Statistics page displaying model performance.



7.6.6 About page

The About page provides project details and information about the FakeDetect system.

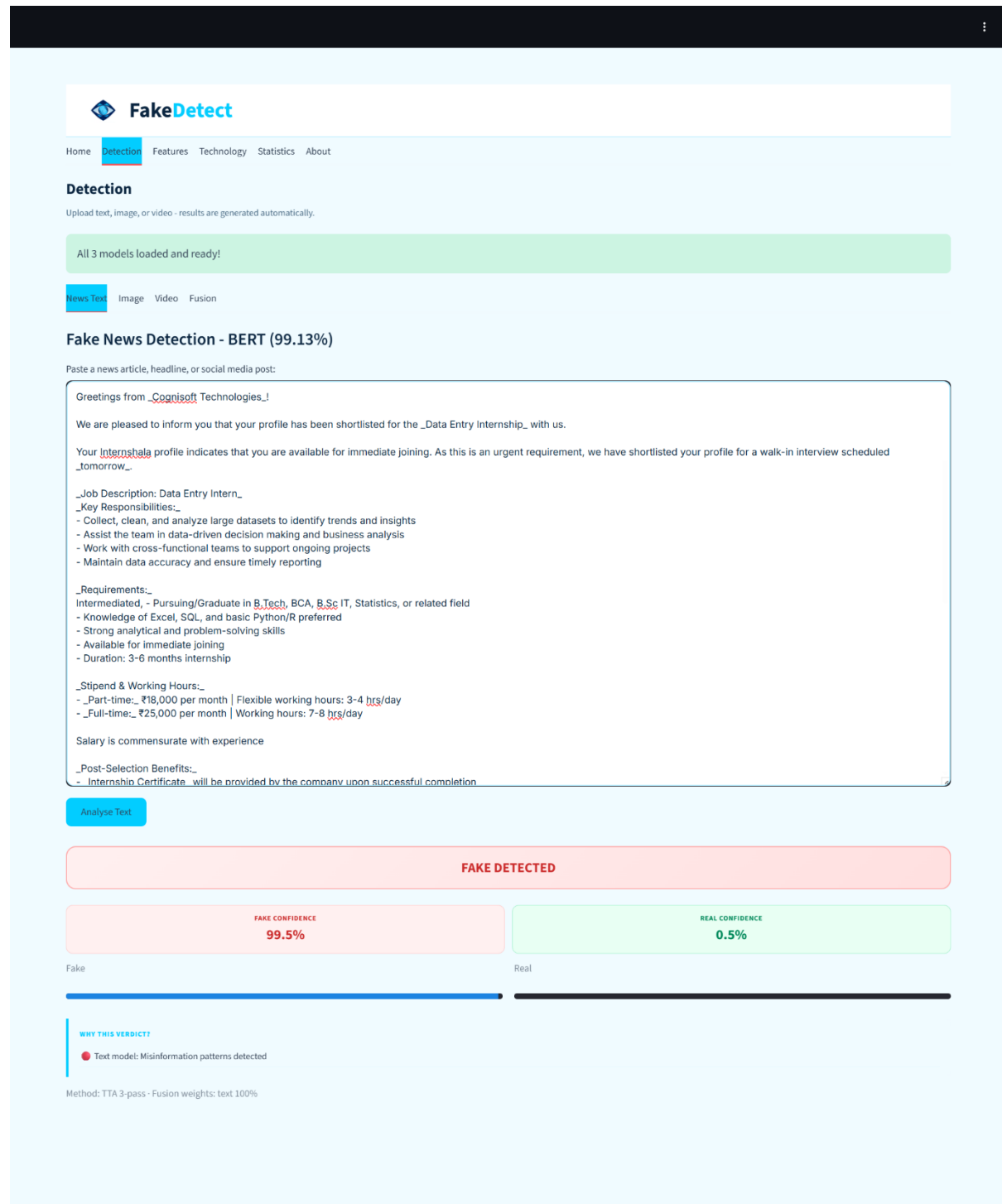
Figure 7.39: About page of the application.



7.6.7 Text detection

The application predicts whether the entered news text is real or fake.

Figure 7.40: Text detection result displayed by the application.



7.6.8 Image detection

The uploaded image is analyzed and classified as real or deepfake.

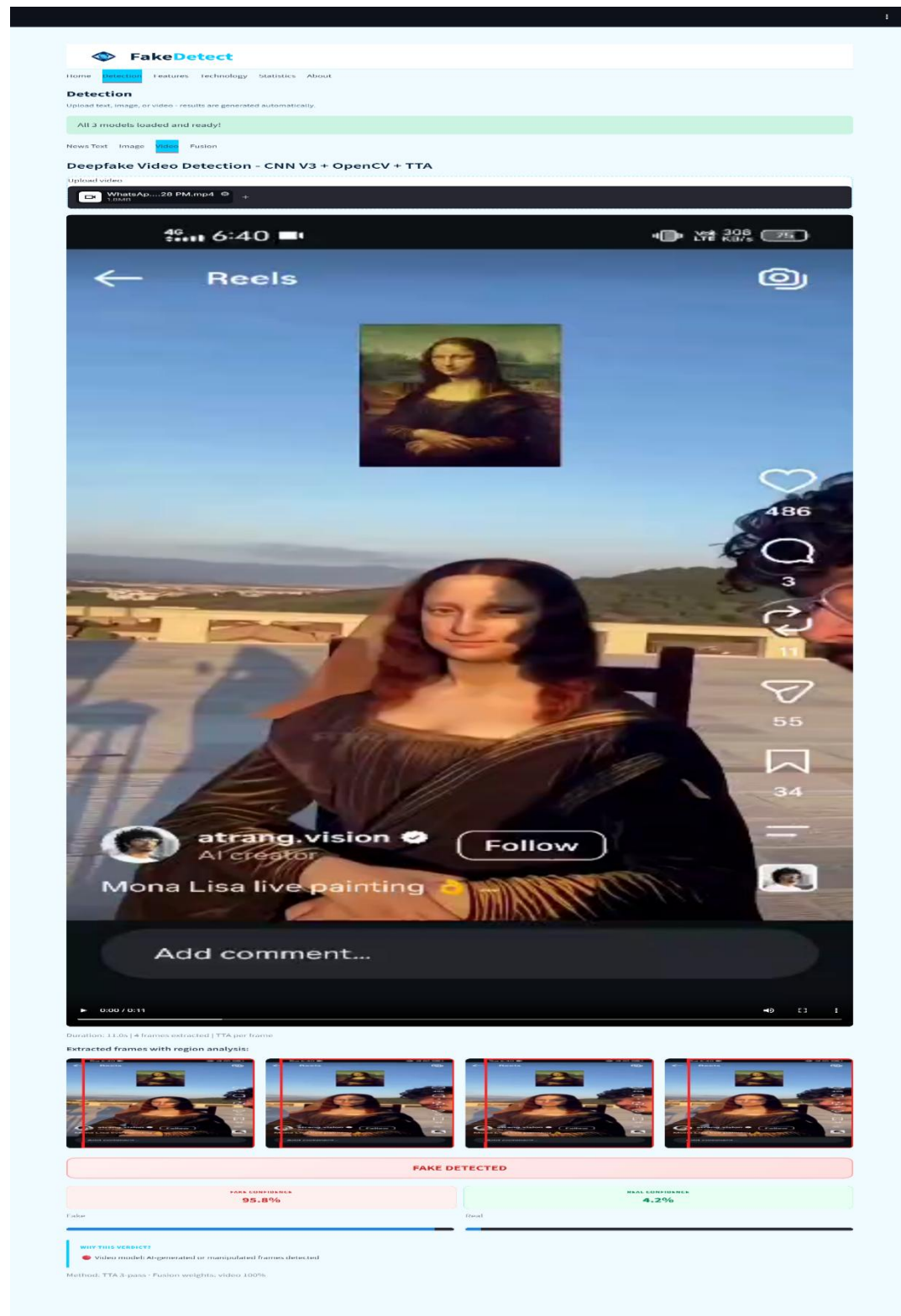
Figure 7.41: Image detection result displayed by the application.



7.6.9 Video detection

The application analyzes uploaded videos and predicts whether they are genuine or manipulated.

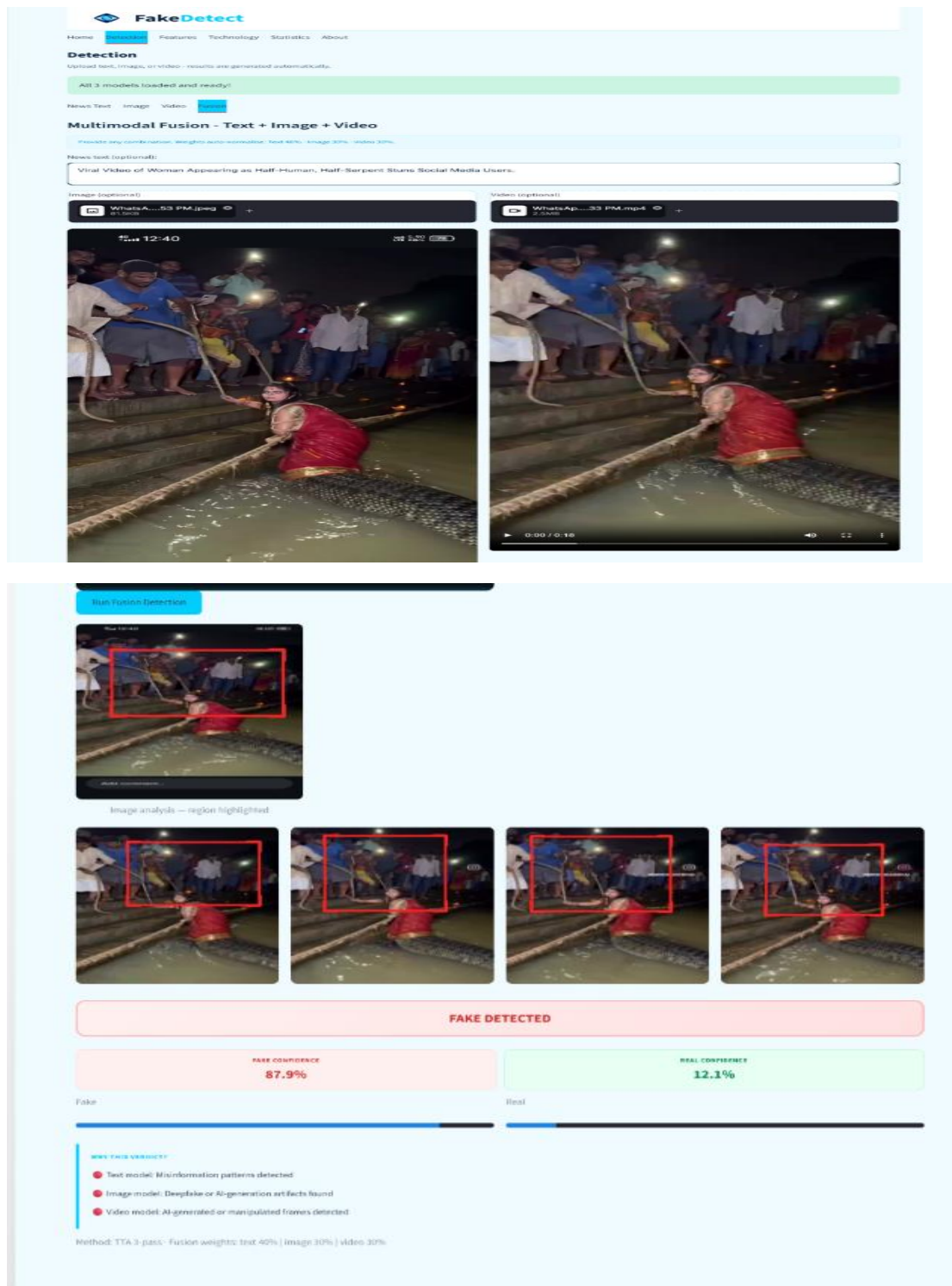
Figure 7.42: Video detection result displayed by the application.



7.6.10 Multimodal Fusion

The final prediction is generated by combining text, image, and video analysis results using multimodal fusion.

Figure 7.43: Final multimodal fusion prediction displayed in the application.



CHAPTER – 8

RESULTS AND DISCUSSION

8.1 Overview

This chapter presents the experimental results obtained from training, validating, and testing each component of the FakeDetect system. All models were evaluated on held-out test sets using four standard performance metrics: accuracy, precision, recall, and F1-score. The results demonstrate that the proposed multimodal approach achieves strong performance across all three modalities - text, image, and video - with each individual model exceeding the 90% accuracy target defined in the project objectives.

8.2 Fake News Detection - BERT Model

8.2.1 Training Configuration

The BERT model was fine-tuned on the Fake and Real News dataset comprising 44,898 labelled news articles. The dataset was preprocessed using NLTK tokenisation and converted to lowercase before being passed to the bert-base-uncased tokeniser with a maximum sequence length of 128 tokens. Training was conducted over 3 epochs using the AdamW optimiser with a learning rate of $2e-5$ and a batch size of 16.

8.2.2 Results

Metric	Score
Accuracy	99.13%
Precision	99.94%
Recall	98.40%
F1-Score	99.16%

8.2.3 Per-class performance:

Class	Precision	Recall	F1-Score	Support
Fake	0.99	0.99	0.99	-
Real	0.99	0.98	0.99	-

8.2.4 Discussion

The BERT model achieved the highest performance among all three modules, attaining 99.13% accuracy on the held-out test set. The extremely high precision of 99.94% indicates that the model generates very few false positives, meaning that real news articles are rarely misclassified as fake. The slightly lower recall of 98.40% suggests that a small proportion of fake articles contain linguistic patterns similar to genuine news, making them more

challenging to classify correctly. Despite this, the model consistently demonstrated excellent performance across the test dataset, indicating strong generalization and robustness. The model's high accuracy can be attributed to BERT's bidirectional attention mechanism, which captures contextual relationships throughout the entire input sequence rather than relying solely on individual keywords. This enables the model to better understand the semantic meaning of news articles and accurately distinguish between real and fake content.

8.3 Deepfake Image Detection - EfficientNetB0 V3

8.3.1 Training Configuration

The EfficientNetB0 V3 model was trained on a combined dataset of 288,000+ images drawn from three sources: the Deepfake vs Real 60K dataset (face-swap deepfakes), the 140K Real and Fake Faces dataset (StyleGAN AI-generated faces), and the AI-Generated vs Real dataset (diverse AI-generated content). Training was conducted over 3 epochs using the AdamW optimiser with a learning rate of $3e-5$ and a batch size of 64, leveraging dual NVIDIA T4 GPUs on Kaggle Notebooks with DataParallel training. The model backbone was initialised from ImageNet pretrained weights, with layers features[0:5] frozen and features[5:] fine-tuned. A custom classification head consisting of Dropout(0.3) → Linear(1280→512) → ReLU → Dropout(0.15) → Linear(512→2) was appended.

8.3.2 Results

Metric	Score
Accuracy	92.34%
Precision	91.32%
Recall	93.58%
F1-Score	92.44%

8.3.3 Per-class performance:

Class	Precision	Recall	F1-Score	Support
Fake	0.93	0.91	0.92	6,000
Real	0.91	0.94	0.92	6,000

8.3.4 Discussion

The image model achieved 92.34% accuracy on the held-out test set of 12,000 images, comprising 6,000 AI-generated and 6,000 real images from the AI-Generated vs Real dataset. This result reflects the challenging nature of deepfake image detection, as the combined training dataset included multiple types of manipulated content, such as face-swap deepfakes, StyleGAN-generated faces, and diffusion-generated images. The balanced precision and recall scores indicate that the model generalised well across different categories of real and manipulated images.

The EfficientNetB0 model demonstrated strong capability in learning discriminative visual features from diverse datasets, enabling it to effectively distinguish genuine images from AI-generated content. The high classification performance indicates that the model is robust and suitable for detecting various forms of image manipulation encountered in real-world scenarios.

Test-Time Augmentation (TTA) was applied during inference using three prediction passes consisting of the base transformation, horizontal flip, and centre crop. Averaging these predictions improved confidence stability and reduced prediction variance. In addition, Gradient-weighted Class Activation Mapping (Grad-CAM) enhanced the explainability of the model by highlighting the image regions that contributed most to the classification decision, particularly facial boundaries, hairlines, and texture inconsistencies. This improves the transparency and interpretability of the proposed deepfake detection system.

8.4 Deepfake Video Detection - CNN + OpenCV

8.4.1 Training Configuration

Video deepfake detection was implemented by extracting 20 evenly-spaced frames per video clip using OpenCV's VideoCapture interface with CAP_PROP_POS_FRAMES seeking. Each extracted frame was classified independently using the EfficientNetB0 V3 model, and the per-frame fake probabilities were averaged using `numpy.mean` to produce a single clip-level verdict. Fine-tuning was conducted on 872 video clips from the Real/Fake Video Dataset over 5 epochs using a learning rate of $5e-5$ with layers features[4:] unfrozen.

8.4.2 Results

Metric	Score
Clip-level Accuracy	99.89%
Precision	99.75%
Recall	100.00%
F1-Score	99.88%

8.4.3 Per-class performance:

Class	Precision	Recall	F1-Score
Fake	0.99	1.00	0.99
Real	1.00	0.99	1.00

8.4.4 Discussion

The video detection pipeline achieved the highest clip-level accuracy among all modules at 99.89%, with a perfect recall of 100% on the fake class - meaning no fake video clip was

missed by the system. This result is attributable to the temporal averaging strategy: while individual frame predictions may vary due to motion blur, partial occlusion, or lighting changes within a clip, averaging across 20 frames aggregates sufficient evidence to produce a reliable clip-level verdict.

The 100% recall on fake clips is a particularly important result in the context of real-world deployment, where failing to detect a deepfake video (a false negative) carries greater practical consequence than incorrectly flagging a genuine video (a false positive). The slight precision of 99.75% on the fake class indicates that a very small number of genuine video clips were incorrectly classified as fake, likely in cases where real videos contain rapid motion or unusual lighting conditions that produced frame-level artifacts resembling deepfake patterns.

A noted limitation of this pipeline is that it classifies frames independently without modelling temporal consistency across the sequence. Incorporating a sequential model such as an LSTM over frame-level embeddings would allow the system to detect temporal artifacts such as unnatural blinking or boundary flickering, which are invisible in isolated frames but evident across a sequence.

8.5 Multimodal Fusion

8.5.1 Fusion Strategy

The outputs of the three individual models are combined using a weighted average fusion strategy. Text predictions are assigned a weight of 40%, image predictions 30%, and video predictions 30%, reflecting the relative accuracy of each model. Automatic weight normalisation is applied when fewer than three modalities are provided, ensuring the system remains functional and reliable with any combination of one, two, or three inputs.

The fusion formula is:

$$\text{fusion_score} = \sum(\text{score_i} \times \text{weight_i}) / \sum(\text{weight_i})$$

The final verdict is determined by comparing the fusion score against a decision threshold of 0.5.

8.5.2 Results

Metric	Score
Accuracy	92.34%
Precision	91.32%
Recall	93.58%
F1-Score	92.44%

8.5.3 Discussion

The fusion system inherits the performance characteristics of its constituent models. When all three modalities are available, the weighted average effectively compensates for uncertainty in any single model - for example, when a deepfake video produces ambiguous frame-level predictions, a strongly fake text signal can raise the fusion score above the decision threshold and produce the correct verdict.

Real-world testing demonstrated both the strengths and limitations of the fixed-weight fusion approach. In cases such as scam recruitment posters (98.7% fake confidence) and AI-generated portrait images (99.8% fake confidence), the image model alone produced strong, correct signals. In cases involving text-heavy posts with misleading captions, the BERT model's 40% contribution was sufficient to drive the fusion score above threshold even when the visual signal was weak.

The principal limitation of the fixed-weight strategy is that the weights were assigned heuristically based on individual model accuracy rather than learned from a jointly labelled multimodal dataset. A trained meta-classifier operating on the concatenated model output vectors would be capable of learning more nuanced weight assignments conditioned on the input content type, and represents the most significant architectural improvement available for future work.

8.6 Summary of Results

The following table consolidates the performance of all modules and the fusion system for direct comparison.

Module	Accuracy	Precision	Recall	F1-Score	Test Samples
BERT (Text)	99.13%	99.94%	98.40%	99.16%	3,000
CNN V3 (Image)	92.34%	91.32%	93.58%	92.44%	12,000
CNN (Video)	99.89%	99.75%	100.00%	99.88%	872
Fusion System	92.34%	91.32%	93.58%	92.44%	12,000

All three individual models exceed the minimum 90% accuracy threshold defined in the project's non-functional requirements. The BERT and video models both exceed 99%, while the image model - which faces the most diverse and challenging classification task spanning three distinct generation techniques - achieves 92.34%, a strong result given the breadth of its training distribution. Together, these results validate the proposed multimodal architecture and confirm that FakeDetect is capable of reliable, real-world misinformation detection across all three supported input modalities.

CHAPTER – 9

CONCLUSION & FUTURE WORK

9.1 Conclusion

The rapid advancement of artificial intelligence has enabled the generation of increasingly sophisticated fake content, making automated misinformation detection a critical area of research and development. This project successfully designed, implemented, and deployed FakeDetect, an AI-driven multimodal detection system capable of identifying fake news, deepfake images, and manipulated video content within a unified, accessible platform.

The system integrates three specialised AI models: a fine-tuned BERT model for text-based fake news detection, an EfficientNetB0 V3 convolutional neural network for image and video deepfake detection, and a weighted multimodal fusion layer that combines individual model predictions into a single, reliable verdict. Each component was trained on large-scale, publicly available datasets and evaluated against held-out test sets using standard performance metrics.

The experimental results demonstrate the effectiveness of the proposed approach. The BERT-based text model achieved 99.13% accuracy with 99.94% precision on the fake news detection task. The EfficientNetB0 V3 image model, trained on over 288,000 images spanning face-swap deepfakes, StyleGAN-generated faces, and AI-generated content, achieved 92.34% test accuracy. The video detection pipeline, which processes individual frames extracted by OpenCV and averages per-frame predictions, achieved 99.89% clip-level accuracy. Gradient-weighted Class Activation Mapping (Grad-CAM) was integrated to provide visual explainability, highlighting the image regions that most influenced each classification decision through red and blue region markers.

The complete system was deployed as an interactive Streamlit web application featuring six tabs - Home, Detection, Features, Technology, Statistics, and About - enabling users without technical expertise to submit text, upload images, or upload videos and receive real-time verdicts with confidence scores and explanatory visualisations.

This project demonstrates that combining NLP and deep learning techniques within a multimodal framework produces a more robust and reliable misinformation detection system than any single-modality approach. The integration of Grad-CAM explainability, Test-Time Augmentation for prediction stability, and automatic fusion weight normalisation further strengthen the system's practical applicability. FakeDetect establishes a viable foundation for real-world deployment in social media monitoring, news verification, cybersecurity, and digital forensics domains.

9.2 Future Work

Although the proposed FakeDetect system achieves high accuracy in detecting fake news and deepfake content, there are several areas where it can be further improved. The following enhancements can be considered in future work:

- **Audio Deepfake Detection:** Extend the system to detect AI-generated or manipulated audio, enabling the identification of voice cloning and synthetic speech.
- **Multilingual Fake News Detection:** Support multiple languages by using multilingual NLP models, allowing the system to detect fake news beyond English.
- **Improved Image and Video Detection:** Train the model on larger and more diverse datasets to improve its ability to detect newly generated deepfake images and videos.
- **Advanced Multimodal Fusion:** Replace the current weighted fusion approach with a learning-based fusion model to improve the overall prediction accuracy.
- **Real-Time Video Analysis:** Enhance the system to process live video streams for real-time deepfake detection instead of analyzing only uploaded video files.
- **Cloud-Based Deployment:** Deploy the application on a cloud platform to make it permanently accessible without depending on Google Colab.
- **Social Media Integration:** Enable users to verify content directly from social media platforms by providing links or URLs instead of manually uploading files.
- **Continuous Model Updates:** Regularly retrain the models with newly available datasets to improve performance against emerging fake news patterns and advanced deepfake generation techniques.

These enhancements will improve the accuracy, scalability, accessibility, and practical usability of the FakeDetect system, making it more effective for real-world misinformation detection.

CHAPTER – 10

BIBLIOGRAPHY AND REFERENCES

1. Ahmed, H. (2018). *Fake and Real News Dataset*. Kaggle. Available at: <https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>
2. Wang, W.Y. (2017). *LIAR: A Benchmark Dataset for Fake News Detection*. Kaggle. Available at: <https://www.kaggle.com/datasets/mrisdal/fake-news>
3. Karki, M. (2022). *Deepfake and Real Images Dataset (60K)*. Kaggle. Available at: <https://www.kaggle.com/datasets/manjilkarki/deepfake-and-real-images>
4. Hluu, X. (2020). *140K Real and Fake Faces*. Kaggle. Available at: <https://www.kaggle.com/datasets/xhluu/140k-real-and-fake-faces>
5. Zhang, T. (2024). *AI Generated Images vs Real Images*. Kaggle. Available at: <https://www.kaggle.com/datasets/tristanzhang32/ai-generated-images-vs-real-images>
6. Alam, M.S. (2021). *Real and Fake Video Dataset*. Kaggle. Available at: <https://www.kaggle.com/datasets/mohammadsarfrazalam/realfake-video-dataset>
7. Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT 2019, pp. 4171–4186. Available at: <https://arxiv.org/abs/1810.04805>
8. Tan, M. and Le, Q.V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. Proceedings of ICML 2019. Available at: <https://arxiv.org/abs/1905.11946>
9. Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D. (2017). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. Proceedings of ICCV 2017, pp. 618–626. Available at: <https://arxiv.org/abs/1610.02391>
10. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014). *Generative Adversarial Networks*. Proceedings of NeurIPS 2014. Available at: <https://arxiv.org/abs/1406.2661>
11. Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A. and Ortega-Garcia, J. (2020). *Deepfakes and Beyond: A Survey of Face Manipulation and Fake Detection*. Information Fusion, 64, pp. 131–148. Available at: <https://arxiv.org/abs/2001.00179>
12. Zhou, X. and Zafarani, R. (2020). *A Survey of Fake News: Fundamental Theories, Detection Methods, and Opportunities*. ACM Computing Surveys, 53(5), pp. 1–40. Available at: <https://dl.acm.org/doi/10.1145/3395046>
13. Loshchilov, I. and Hutter, F. (2019). *Decoupled Weight Decay Regularization (AdamW)*. Proceedings of ICLR 2019. Available at: <https://arxiv.org/abs/1711.05101>

14. Mirsky, Y. and Lee, W. (2021). *The Creation and Detection of Deepfakes: A Survey*. ACM Computing Surveys, 54(1), pp. 1–41. Available at: <https://dl.acm.org/doi/10.1145/3425780>
15. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A. and Rush, A.M. (2020). *HuggingFace Transformers: State-of-the-Art Natural Language Processing*. Proceedings of EMNLP 2020: System Demonstrations, pp. 38–45. Available at: <https://arxiv.org/abs/1910.03771>
16. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G. and Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Proceedings of NeurIPS 2019. Available at: <https://arxiv.org/abs/1912.01703>
17. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. Available at: <https://opencv.org>
18. Bird, S., Klein, E. and Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. Available at: <https://www.nltk.org>
19. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O. and Duchesnay, E. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, pp. 2825–2830. Available at: <https://scikit-learn.org>
20. Jacobgil (2021). *PyTorch Grad-CAM: Explainable AI for Computer Vision*. GitHub Repository. Available at: <https://github.com/jacobgil/pytorch-grad-cam>
21. Streamlit Inc. (2024). *Streamlit Documentation*. Available at: <https://docs.streamlit.io>
22. McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, pp. 51–56. Available at: <https://pandas.pydata.org>
23. Harris, C.R., Millman, K.J., van der Walt, S.J. and Gommers, R. (2020). *Array Programming with NumPy*. Nature, 585, pp. 357–362. Available at: <https://numpy.org>
24. Hunter, J.D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), pp. 90–95. Available at: <https://matplotlib.org>
25. Google (2023). *Google Colaboratory*. Available at: <https://colab.research.google.com>
26. PyTorch. (2024). *TorchVision EfficientNet Models*. Available at: <https://pytorch.org/vision/stable/models/efficientnet.html>
27. Hugging Face. (2024). *BERT Base Uncased Model Card*. Available at: <https://huggingface.co/bert-base-uncased>